



SCHOOL OF COMPUTER SCIENCE

Using EEG Spectrograms for Natural Language Processing and Part of Speech Classification

Gregory Seaton

A dissertation submitted to the University of Bristol in accordance with the requirements of the degree
of Bachelor of Science in the Faculty of Engineering **worth 40CP**.

Monday 28th April, 2025

Abstract

This paper investigates a neural network-based approach for extracting linguistic patterns from EEG data. I was able to structure a neural network that, given spectrogram inputs of EEG data recorded while an individual was reading words on a screen[59], was able to predict the corresponding GloVe [44] word embedding with a high degree of accuracy, measured by cosine similarity 0.6741. Upon using other evaluation metrics, however, the meaning of this figure comes into question, prompting a discussion on the suitability of cosine similarity for such tasks. A similar architecture, with some minor tweaks in neural network structure, was also used to classify content vs function words with an accuracy of 66.01%. Using data recorded from predicting word embeddings, I also attempt to support current research in the field of linguistics discussing the degree to which Part of Speech is embedded in natural human linguistics, as opposed to being a syntactical construct of modern languages. As well as this, I explore the GloVe word embedding space by running clustering algorithms on it, to reveal greater insight into its underlying structure, alongside making content vs function predictions on word embedding vectors, both predicted and true.

The research in this project is divided into 4 main sections for Methodology, Results and Critical Evaluation:

- Improving efficiency and accuracy of POS classifier (content/function).
- Predicting word embeddings from EEG data.
- Clustering in word embedding space.
- Using predicted word embeddings for content/function classification.

Dedication and Acknowledgments

I would like to extend huge thanks to my supervisor, George Sains for his support and advice in conducting this research project. His guidance throughout the project was invaluable, both in refining my ideas and in overcoming the challenges I encountered.

I would also like to thank my family for supporting me through my time at university, spurring me on to grow and develop. I am deeply grateful for their unwavering support throughout this journey.

Declaration

I declare that the work in this dissertation was carried out in accordance with the requirements of the University's Regulations and Code of Practice for Taught Programmes and that it has not been submitted for any other academic award. Except where indicated by specific reference in the text, this work is my own work. Work done in collaboration with, or with the assistance of others including AI methods, is indicated as such. I have identified all material in this dissertation which is not my own work through appropriate referencing and acknowledgement. Where I have quoted or otherwise incorporated material which is the work of others, I have included the source in the references. Any views expressed in the dissertation, other than referenced material, are those of the author.

Gregory Seaton, Monday 28th April, 2025

AI Declaration

I declare that any and all AI usage within the project has been recorded and noted within Appendix A or within the main body of the text itself. This includes (but is not limited to) usage of text generation methods incl. LLMs, text summarisation methods, or image generation methods.

I understand that failing to divulge use of AI within my work counts as contract cheating and can result in a zero mark for the dissertation or even requiring me to withdraw from the University.

Gregory Seaton, Monday 28th April, 2025

Contents

1	Introduction	1
2	Background	3
2.1	Explanation of Electroencephalogram (EEG) and Spectrograms	3
2.2	Literature Review	3
2.3	Word Embeddings Models	5
2.4	Machine Learning Technologies used	7
3	Project Execution	16
3.1	Methodology	16
4	Results and Critical Evaluation	23
4.1	POS classifier	23
4.2	Predicting word embeddings	24
4.3	Clustering in Word Embedding Space	26
4.4	Predicted Word Embeddings for Content Function Classification	30
5	Conclusion	34
A	Appendix A: AI Prompts/Tools	40
B	Raw Test Results	41

List of Figures

2.1	Initial CNN architecture used in the project	5
2.2	A simple visual representation of a decision tree used to classify dog and cat from [36] . .	9
2.3	A flowchart to represent how Decision Trees are trained. Top and bottom nodes reference:[36], middle node reference:[1]	10
2.4	An example of two clusters being identified from a set of data points[34]	11
3.1	Demonstration of Colour to Grayscale Spectrogram Conversion	17
4.1	POS tags the average verb is most likely to get confused for	25
4.2	POS tags the average noun is most likely to get confused for	26
4.3	Silhouette Score (left) and Davies-Bouldin Score (right) plotted against Number of Clusters, k	26
4.4	DBScan <i>min_samples</i> vs <i>eps</i> HeatMaps using Euclidean Distance. Top-left: Number of Clusters. Top-right: Proportion Data classified as Noise. Bottom-left: Silhouette Score, Bottom-right: Davies-Bouldin Score	27
4.5	DBScan <i>min_samples</i> vs <i>eps</i> HeatMaps using Cosine Distance. Top-left: Number of Clusters. Top-right: Proportion Data classified as Noise. Bottom-left: Silhouette Score, Bottom-right: Davies-Bouldin Score	28
4.6	HDBScan Graphs using Euclidean Distance. Plotting <i>min_cluster_size</i> against Number of Clusters (top-left), Proportion of Noise (top-right), Silhouette Score (bottom-left), Davies Bouldin Score (bottom-right)	29
4.7	HDBScan Graphs using Cosine Distance. Plotting <i>min_cluster_size</i> against Number of Clusters (top-left), Proportion of Noise (top-right), Silhouette Score (bottom-left), Davies Bouldin Score (bottom-right)	30

List of Tables

2.1	Accuracy results for 10 samples[77]	5
2.2	Co-occurrence Matrix with Context Window Size 2	6
2.3	Co-occurrence probability table from [44] (Table 1)	7
3.1	Grid-search hyper-parameters for SVM	21
3.2	Grid-search hyper-parameters for MLP classifier	21
3.3	Grid-search hyper-parameters for Decision Tree classifier	22
3.4	Grid search parameters for AdaBoost	22
4.1	Models to Maximise Efficiency Training Times and Accuracy	23
4.2	Model to Maximise Accuracy Training Times and Accuracy	24
4.3	Cosine similarities between pairs of words	24
4.4	Cosine Similarity Results	31
4.5	SVM Grid Search Results	31
4.6	Neural Network Grid Search Results	31
4.7	Decision Tree Grid Search Results	31
4.8	Random Forest Grid Search Results	32
4.9	AdaBoost Grid Search Results	32
4.10	Best parameters and models from previous tests	32
4.11	Voting and Stacking Accuracy Comparison	32
B.1	Accuracy and Duration for 10 runs of Second Model from 4.1	41

Ethics Statement

This project did not require ethical review, as determined by my supervisor, George Sains.

Supporting Technologies

- Optuna: Used to optimise neural network hyperparameters
- TensorFlow: Python library used for neural networks
- Sci-kit learn: Python library used for other machine learning techniques
- MATLAB: Used to generate spectrogram images
- BluePebble: Supercomputer to offload computation to
- GloVe: Pretained word embedding model ued.

Chapter 1

Introduction

Huge progress has been made in the field of Natural Language Processing (NLP) in recent years[67], to the extent that machines can accurately understand and convincingly respond to and act upon human language inputs. Looking into the future, it is almost inevitable that this powerful technology will become an ever more prominent part of our daily lives, and the way we interface with it is likely to change accordingly.

At the moment, we type our prompts into an LLM, after which they are processed to uncover the semantics behind the syntactical representation of our thoughts. I believe natural progression of this would be to bridge the syntactical gap by allowing direct access to record our neural activity. This would introduce a number of advantages compared to the current approach of interacting with an LLM:

- Less computational overhead: No need to decipher syntax if you can directly extract semantics.
- Faster communication: The user does not need to spend time articulating themselves.
- More accurate communication: Could resolve a kind of “Gulf of Expression”, bridging the gap between what someone is capable of articulating, and what they actually want to convey.

Such developments could go so far as constant (potentially subconscious) communication between a user and a machine, vastly expanding their mental capabilities.

Though this research project is not claiming a breakthrough of this magnitude, I believe it is important to gather as much understanding as possible of this technology as it matures, to help highlight the issues that the industry will have to confront in the coming years.

I aim to explore the foundations of this field using a dataset of electroencephalogram (EEG) data recorded by 32 nodes placed on different parts of the scalp [59], while participants read different words and sentences. I will aim to extract linguistic information from these recordings, specifically Part-of-Speech (POS) tagging and quantifying the semantics of words being read, by trying to predict their word embedding.

This project aims to fill a gap in current literature by directly predicting word embedding vectors from brain wave activity, measured by EEG nodes.

I also believe that analysis of my results may be able to contribute to linguistic research into the degree to which different Parts of Speech are hardwired into human cognition, as opposed to being a product of modern language.

The high-level objective is to use this EEG recording dataset to:

- Improve the accuracy and efficiency of a previous attempt [77] at POS tagging.
- Predict word embedding vectors

1.0.1 Progression of the Project

The initial goal of this project was to improve the accuracy and efficiency of the POS (Part-of-Speech) classifier produced in a previous research project [77]. It was designed to take inputs from a dataset of EEG recordings taken while participants were reading words on a screen [59]. The classifier was able to classify content vs function words with some degree of accuracy, but was not very efficient. Despite this, the project set out a very effective methodology for using neural networks to analyse raw EEG data. Upon achieving the initial goals set out, I used this methodology to expand the scope of the project further.

One potential path to explore would be to expand the number of POS classes to try and predict, but I decided against this, as the best accuracy I could achieve for the simplest POS classification task (content vs function) was quite poor. This motivated me to explore why this could be. I considered that POS might be purely syntactic, meaning it may be harder to classify given just brain wave data (this is a thoroughly discussed topic in the field of linguistics - in reality, it is likely something in between). An anecdote that could support the argument that POS is syntactical is to consider two related words, such as “food” and “eat”. Despite belonging to different classes, these words are strongly related, and may be processed by the brain in similar ways. On the other hand, despite belonging to the same class, “kettle” and “underpass” are extremely unrelated and may not be processed in the same way.

This led me to try using the same neural network structure to predict the actual semantics of words. Word embeddings can be used to quantify the semantics of language, as covered in the foundational article [68]. In this approach, words are assigned a vector, giving them a location in hyperdimensional vector space, where similar words are positioned closer together. This makes up a major part of this project, where I attempt to predict word embeddings. My work on this is covered extensively in the Methodology and Analysis sections of this report.

Rather than predicting word embeddings directly, another method could be to calculate clusters, and assign words to these clusters by one-hot-encoding. This could potentially reveal some kind of underlying structure, that may be hardwired into the brain, but this approach raises a number of issues. My attempt at this and the issues that arose is covered in the Methodology section of this project.

Chapter 2

Background

2.1 Explanation of Electroencephalogram (EEG) and Spectrograms

Electroencephalogram is a non-invasive technique for recording the electrical activity produced by the brain. Electrodes which detect voltage changes are placed on the scalp. Such voltage changes are caused by the ionic flow of neurons in the brain, as a result, to some extent, they can represent brain wave activity[74]. Voltage oscillations, measured as frequency, represent different types of brain wave activity, with higher frequencies typically represent conscious activity, and smaller frequencies representing subconscious activity[53].

Spectrograms are used to visually represent time-frequency data, with different colours (or greyscale values) representing the prominence of different frequencies[49]. Though this, spectrograms reveal how different frequencies change over time, providing a powerful visual representation of the brain’s activity.

2.2 Literature Review

2.2.1 Existing Research on the use of EEG data

Numerous studies have used machine learning to process EEG data by converting it to spectrograms. Some studies also attempt part-of-speech classification and sentiment analysis from EEG. This project fills a gap in the literature by combining these two concepts, particularly by testing on words that were not necessarily in the training set, as well as predicting actual word embedding vectors.

The methodology of converting EEG to spectrograms for use in a CNN is demonstrated by *Datta and Boulgouris (2021)*[37]. The authors attempt to classify whether the participant was reading a verb or a noun with a remarkable accuracy of 85.7% on unseen words. This success highlights the efficacy of this methodology, where spectrograms are generated from raw EEG data, which are then passed through a CNN. It also demonstrates that this can be done independently of the subject, i.e., by testing on a subject whose readings were not part of the training set, achieving an accuracy of 78.5%.

Mohan and Anand (2025) [51] demonstrate that EEG data can be used to classify specific words. The authors attempt to classify EEG readings from a small selection of words, all of which the model is both trained and tested on. The methodology involves using their novel strategy “imagined speech functional connectivity graph” (ISFCG), which forms graphs that capture complex relationships between brain regions, while the participants imagine certain words. These graphs are then fed into a CNN. It successfully captures spatial and relational information between brain regions to achieve the following results:

- 0.71 accuracy classifying long words: “independent” and “cooperate”
- 0.45 accuracy classifying short words: “in”, “out”, and “up”

- 0.46 accuracy classifying vowels: “/a/”, “/i/”, “/u/”
- 0.7 accuracy classifying short-long words: “cooperate” and “in”

These results demonstrate a potential challenge in classifying words used in similar contexts. The accuracy achieved on short words is relatively poor, being only slightly above the 0.33 baseline on three classes. This could be because all the words are prepositions, which, though they have different meanings, can be used in similar contexts and likely require similar communication between different brain regions to process, suggesting that focusing on intra-brain communication may be less effective.

The ability to draw linguistic semantics from EEG data is demonstrated by Hollenstein et al. (2021)[56]. In this paper, the authors demonstrate that after extracting features from EEG signals, they can be combined with word embeddings from pre-trained models to significantly improve sentiment classification performance compared to using EEG data or word embeddings alone. As a result, it can be deduced that semantic information can be drawn from EEG data.

2.2.2 Existing Research on whether POS is syntactical or hardwired

Summary of Current Viewpoints

The debate on whether Part of Speech labels (in English, nouns, verbs, adjectives, pronouns etc.) are syntactical products of modern languages or hardwired into human cognition, is focused on two opposing viewpoints. This is only a summary of a decades-long debate, and does not fully represent both sides of the argument.

The nativist viewpoint, spearheaded by the work of Noam Chomsky asserts that the ability to understand grammar is hardwired into the human brain, allowing children to learn specific languages remarkably quickly because the foundations of grammar are already established [48]. This is supported by the “poverty of stimulus” argument [47], which stresses that the linguistic input received in the developmental stages of childhood is typically limited and fragmented, with errors in understanding not being explicitly corrected. Nativists argue that because of this, language acquisition would be impossible without an existing understanding of grammar.

The opposing, constructivist viewpoint emphasises the role of environmental input and cognitive mechanisms for the acquisition of language [28]. Constructivists argue that children learn language by observing patterns and regularities in the language they are exposed to, stressing that grammatical categories are not innate, but are learned through this experience.

Relevant research drawn from EEG data

Carolina Yudes et al. (2016) [31] use EEG data to explore whether parts of speech are syntactic or hardwired in human cognition. Focusing on verbs and nouns, the authors found that the brain differentiates between these classes within 200ms of the stimulus. Notably, the study was conducted in Spanish, where many words can switch between verb and noun forms through simple suffix changes. Despite their similar semantics and surface forms (i.e. the words look similar), the brain still distinguishes them, highlighting the cognitive sensitivity to grammatical class. As stated on page 114, the authors observe that this verb-noun distinction occurred before any significant effects of “Semantic Attribute” were observed, suggesting that the brain may distinguish certain grammatical classes, before processing semantics.

2.2.3 Previous Research Project [77]

This research project continues from a previous project, which aimed to classify content vs function words. It uses a dataset where EEG recordings were taken from 32 nodes while participants read words on a screen. The project set out a very effective preprocessing methodology where spectrograms are generated for each of the 32 EEG nodes. This is a crucial step as it allows EEG data to be processed without requiring thousands of input nodes. These spectrograms are then fed into a convolutional neural network (CNN), which achieves a final accuracy of 73.33

Żymelka(2024) [77] extensively tested the efficacy of numerous spectrograms, concluding that no window overlap and creation using a square function produced the greatest accuracy. Additional preprocessing steps were also taken to remove noise and frequencies that contain irrelevant information, such as muscle movements. This is done with low-pass filtering to remove high-frequency noise above 40 Hz. After this, a log transform compresses large values which are likely to be artifacts, while emphasising smaller, more meaningful changes.

Despite achieving a high accuracy of 73.33%, this was taken from one of 10 runs, where the other results were significantly worse, suggesting that the execution was not particularly robust, with accuracy standard deviation for the 10 runs of this in Figure 2.2.3 model at 8.3028.

Sample	Accuracy (%)
1	46.66
2	73.33
3	46.66
4	53.33
5	46.66
6	44.99
7	48.33
8	41.66
9	51.66
10	51.66

Table 2.1: Accuracy results for 10 samples[77]

These poor results may be because the input data might not have been properly sorted (this will be elaborated on later).

This graph visualises the structure of the CNN produced by Żymelka from the file `Final_Code_2471654 > Code > CNN_implementation.py`



Figure 2.1: Initial CNN architecture used in the project

2.2.4 Conclusion of Literature Review

This literature review demonstrates that it is possible to draw real linguistic semantics from EEG data. I expand on this by attempting to predict word embedding vectors as accurately as possible, a pursuit I was unable to find existing published work on. Analysis of which types of words are easiest to predict can also be used to contribute to current research on which part-of-speech tags (and the extent to which) are hardwired into human linguistic processes

2.3 Word Embeddings Models

Word embedding models aim to quantify the semantics of words by representing them as vectors. They capture intricate relationships between words based on how they are distributed within vast amounts of text. In this context, they are referred to as "pre-trained" because their word embedding vectors have already been produced. Here I will explain the fundamentals of how two of the main word embedding models are trained, word2vec and GloVe.

2.3.1 word2vec[68]

This model is trained by restructuring a neural network to take words as input. The following is a summarised explanation, where V is the length of the model's vocabulary, and N is the length of the word embedding vector.

- Input layer. Each training instance (a single word) is one-hot encoded into a vector of length V . This means that all elements are 0 except for a 1 that corresponds to the position of the relevant word.
- Hidden layer. This vector is then multiplied by a weight matrix of shape $[V, N]$ (projecting from V to N), but because of the one-hot encoding, only one row of length N is selected, the vector represented by this row is the word embedding vector.
- Output layer (pre-softmax). This embedding vector is then multiplied by a second weight matrix of shape $[N, V]$ (projecting back from N to V). This provides a score for each word that reflects how likely it is to be in the context of that word.
- Softmax layer. These scores (length V) are then passed through the softmax function to give a probability distribution. We can think of this as a probability distribution of words likely to appear around or in place (is "in place" actually the case) of the target word.
- Training via backpropagation. This predicted probability distribution can be measured against the actual probability distribution using a loss function (typically cross-entropy), allowing us to backpropagate to train and converge on optimal weights (and matrix values).

[64][55]

The final word embeddings are taken from the matrix connecting the Input to the Hidden layer, where each row corresponds a word embedding vector in the vocabulary.

2.3.2 GloVe[44]

This word embedding model is trained very differently to word2vec, instead of predicting embeddings by extracting features from local context, GloVe takes a count-based approach that analyses the global properties of the training set.

The co-occurrence matrix is at the core of how this model understands how words are used. It is formed by recording how often one words appears in the context of another. Context is defined by a context window, where two words are considered to be in the context of one another if they are "Window Size" places away from each other, see Figure 2 in *J. Pennington et al. (2014)* [44] for a performance breakdown with different context windows. This will form a symmetric matrix, an example of which is displayed in 2.2, which shows the matrix for the two sentences "The frog hops into the pond" and "The kangaroo hops across the bush", with a context window of size 2.

Table 2.2: Co-occurrence Matrix with Context Window Size 2

	the	frog	hops	into	pond	kangaroo	across	bush
the	0	1	1	1	1	1	1	1
frog	1	0	1	0	0	0	0	0
hops	1	1	0	1	0	1	1	0
into	1	0	1	0	1	0	0	0
pond	1	0	0	1	0	0	0	0
kangaroo	1	0	1	0	0	0	1	0
across	1	0	1	0	0	1	0	1
bush	1	0	0	0	0	0	1	0

In real applications, when trained on a larger corpus than two sentences, this will be a huge $V \times V$ matrix. They are also typically trained with a larger context window size.

From this co-occurrence Matrix, GloVe derives meaning by considering the ratios of co-occurrences. Here is the example given in the original GloVe paper.

Consider two words i and j that exhibit a particular aspect of interest; for concreteness, suppose we are interested in the concept of thermodynamic phase, for which we might take $i = \text{ice}$ and $j = \text{steam}$. The relationship of these words can be examined by studying the ratio of their co-occurrence probabilities with various probe words, k . For words k related to ice but not steam, say $k = \text{solid}$, we expect the ratio P_{ik}/P_{jk} will be large. Similarly, for words k related to steam but not ice, say $k = \text{gas}$, the ratio should be small. For words k like water or fashion, that are either related to both ice and steam, or to neither, the ratio should be close to one. 2.3.2 shows these probabilities and their ratios for a large corpus, and the numbers confirm these expectations. Compared to the raw probabilities, the ratio is better able to distinguish relevant words (solid and gas) from irrelevant words (water and fashion) and it is also better able to discriminate between the two relevant words. [44]

Probability and Ratio	$k = \text{solid}$	$k = \text{gas}$	$k = \text{water}$	$k = \text{fashion}$
$P(k \text{ice})$	1.9×10^{-4}	6.6×10^{-5}	3.0×10^{-3}	1.7×10^{-5}
$P(k \text{steam})$	2.2×10^{-5}	7.8×10^{-4}	2.2×10^{-3}	1.8×10^{-5}
$P(k \text{ice})/P(k \text{steam})$	8.9	8.5×10^{-2}	1.36	0.96

Table 2.3: Co-occurrence probability table from [44] (Table 1)

This quotation suggests that we can use co-occurrence probabilities, formed by the ratios between them, as the foundation for our word embeddings. The GloVe model learns word vectors by capturing these relationships mathematically, encoding how likely words are to appear together. It does this by minimizing a loss function:

$$J = \sum_{i,j} f(X_{ij}) \cdot (w_i^\top \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij})$$

Equation from *Jeffrey Pennington et al. (2014)*[44].

- X_{ij} is the number of times word j appears in the context of word i .
- The vectors w_i and $\tilde{w}_j$ are the word and context embeddings.
- b_i and $\tilde{b}_j$ are bias terms
- The function $f(X_{ij})$ reduces the impact of very frequent/infrequent co-occurrences.

The result of minimising this loss function is a set of vectors w_i , which are the word embedding vectors associated with each word in the vocabulary.

2.4 Machine Learning Technologies used

The following sections will describe the machine learning technologies employed in this project that were not addressed in the Background section of the project this continues from [77].

2.4.1 Support Vector Machines (SVMs)

Support Vector Machines (SVMs) are supervised machine learning models primarily used for classification tasks. The goal of an SVM is to find the optimal hyperplane (decision boundary) that separates data points belonging to different classes $y_i \in \{-1, +1\}$ in a high-dimensional space. The term "Support Vectors" refers the vector data points that define this boundary [35].

Let us consider a dataset of n training examples

$$\{\mathbf{x}_1, \dots, \mathbf{x}_n\} \subset \mathbb{R}^d,$$

where each input vector $\mathbf{x}_i \in \mathbb{R}^d$ is associated with a label $y_i \in \{-1, +1\}$. The decision boundary is defined by the hyperplane

$$\mathbf{w}^\top \mathbf{x} + b = 0,$$

where:

- $\mathbf{w} \in \mathbb{R}^d$ is the weight vector perpendicular to the hyperplane,
- $b \in \mathbb{R}$ is the bias term, which offsets the hyperplane from the origin,
- $\mathbf{x} \in \mathbb{R}^d$ is a data point.[\[30\]](#)

A data point $\mathbf{x}$ is classified based on the result of plugging into:

$$\mathbf{w}^\top \mathbf{x} + b.$$

If this value is $\geq +1$, the point is assigned to class +1; if ≤ -1 , it is assigned to class -1. The region between the hyperplanes

$$\mathbf{w}^\top \mathbf{x} + b = \pm 1$$

is the margin, whose width is

$$\frac{2}{\|\mathbf{w}\|}.$$

Maximising this margin improves the classifier's generalisation. However, real-world data are often not perfectly separable. To allow for some classification errors, soft-margin SVMs introduce slack variables $\xi_i \geq 0$:

- $\xi_i = 0$ if $\mathbf{x}_i$ is correctly classified and lies outside the margin,
- $0 < \xi_i \leq 1$ if $\mathbf{x}_i$ is correctly classified but within the margin,
- $\xi_i > 1$ if $\mathbf{x}_i$ is misclassified.

The optimisation problem then becomes

$$\min_{\mathbf{w}, b, \xi} \quad \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i \tag{2.1}$$

$$\text{subject to} \quad y_i (\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0, \quad i = 1, \dots, n. \tag{2.2}$$

Here, $C > 0$ controls the trade-off between margin width and total slack:

- A small C yields a wider (softer) margin by penalising slack less,
- A large C yields a narrower (harder) margin by penalising slack more.

Dual Representation

There is, however, a slightly more useful way of formulating the decision boundary — this is known as the *dual form*, which replaces our weight vector $\mathbf{w}$ with a weighted sum of the training data points.

By introducing a Lagrange multiplier [\[35\]](#) α_i for each training example, we can rewrite the classification function as

$$f(\mathbf{x}) = \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i^\top \mathbf{x} + b$$

In practice, only the support vectors have non-zero α_i values. These are the points that lie on or inside the margin and decision boundary. The fact that all other α_i are zero highlights how SVMs focus only on the most informative data points when making decisions.

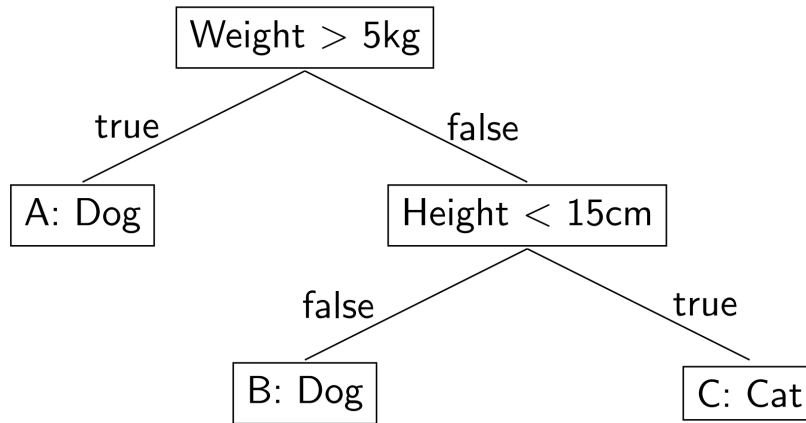


Figure 2.2: A simple visual representation of a decision tree used to classify dog and cat from [36]

Kernel Trick

The true usefulness of the dual representation becomes apparent with the introduction of the *kernel trick*. While the strategy explained above handles linearly separable data well, many real-world problems require non-linear decision boundaries. To address this, SVMs can employ the kernel trick, which implicitly maps inputs $\mathbf{x}$ into a higher-dimensional feature space $\phi(\mathbf{x})$, without ever computing $\phi(\mathbf{x})$ explicitly. This avoids the costly computation of working directly in high dimensions[35].

Instead, the dot products required in the dual representation can be replaced by a kernel function $K(\mathbf{x}_i, \mathbf{x}_j)$, such as the polynomial or radial basis function (RBF) kernel. This allows the SVM to efficiently learn complex, non-linear boundaries.

To enforce how useful it is to be able to handle non linearly separable data, this trick allows SVMs to efficiently form a decision boundary around data points belonging to one class, even if they are completely surrounded by another essentially forming an "island".

Classification for more than two classes

While SVMs are inherently binary classifiers, they can be extended to handle multi-class classification by combining multiple together. One approach is *one-vs-rest*, where a separate classifier is trained to distinguish one class from all others, requiring in n classifiers for n classes. A data point is then assigned to the class whose classifier outputs the highest confidence, measured by each decision function value.[35]

Another common strategy is *one-vs-one*, where a separate classifier is trained for every possible pair of classes, leading to $\frac{n(n-1)}{2}$ classifiers in total. The final prediction is made by a majority vote[35].

It is worth mentioning, however, that while both approaches are effective, the number of classifiers — and thus the computational cost — increases significantly as the number of classes grows. This makes SVMs less practical for problems involving a large number of classes[35].

2.4.2 Decision Trees

Decision trees split the feature space based off feature values, where each leaf node represents a predicted class. This methodology of splitting the feature space by essentially repeatedly asking higher or lower (or true or false) questions for feature values, allows them to model non-linear relationships across many dimensions effectively to form binary trees[36]. Figure 2.3 shows an example.

How they are trained

Decision Trees are trained using a very thorough, greedy algorithm. Starting from the root node:

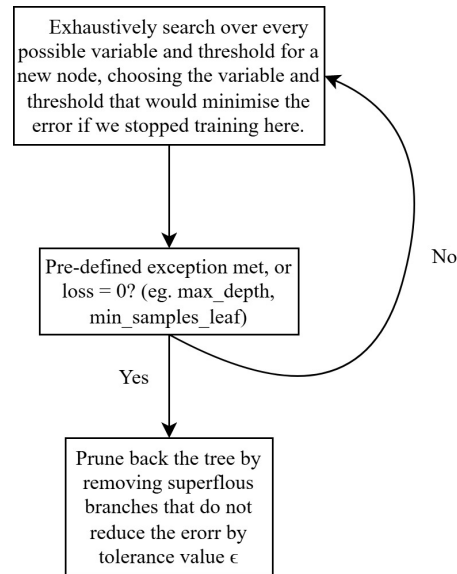


Figure 2.3: A flowchart to represent how Decision Trees are trained. Top and bottom nodes reference:[36], middle node reference:[1]

Proneness to Over-fitting

Without constraints, a tree can perfectly fit the training data, but doing so is likely to make it poorly generalisable, sensitive to small fluctuations in the training set that may just be noise and irrelevant patterns.

Overfitting can be minimised with early stopping criteria, some of which were mentioned in the outline above. Below, they are listed using their corresponding `scikit-learn` hyperparameter names[1]:

- **max_depth**: Limits the depth of the tree. ($\log_2(\text{number of nodes})$), which keeps the model from growing too complex.
- **min_samples_leaf**: Specifies the minimum number of samples required to be at a leaf node. This prevents the model from creating leaves that over-fit tiny subsets of the training data.
- **min_impurity_decrease**: A node will be split only if the resulting decrease in impurity is at least this threshold, helping ensure that only meaningful splits are made.

2.4.3 Ensemble Methods[2]

Ensemble methods are a selection of machine learning techniques that combine multiple models hoping to make better predictions than individual model by themselves. The core principle is to leverage certain models' strengths, while disregarding poorer results given when they fall short[2]. Below are several common ensemble strategies:

2.4.4 Bagging and Boosting

Bagging and Boosting are each categories of Ensemble methods. The former builds multiple models independently and aggregates their outputs, which reduces variance and increases robustness to outliers. Boosting builds models sequentially, where each one focuses on the errors of its predecessor reducing bias.[70]

Random Forests[14]

This implementation of Bagging to the Decision Tree model outlined above. Multiple trees are trained on different subsets of the data. Their outputs are averaged, which reduces overfitting and improves

generalisation.

Adaboost[3]

Adaboost boosting method that combines several "weak learners" (typically shallow decision trees). It makes each model sequentially focusing more on the errors made by the previous ones, adjusting weights to perform better on previously misclassified samples.

Voting[17]

A method that takes the outputs of multiple different models to make a final prediction by a majority vote.

Stacking[15]

Similarly to voting, this technique takes the outputs of several base models. These results can then be fed into a meta-model (often Logistic Regression[15]), which learns how to best combine their predictions.

2.4.5 Clustering Methods

Clustering is a category of unsupervised Machine Learning models, which given unlabelled data, aim to find a pre-specified number of classes. As the name suggests it does this by attempting to separate the data points into clusters. See figure for a visual aid 2.4. The following sections will outline different methods of identifying such clusters, evaluating their advantages and disadvantages. I will use k to represent the number of clusters (defined prior to training the model).

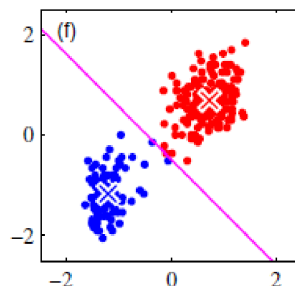


Figure 2.4: An example of two clusters being identified from a set of data points[34]

K-means

K-means is one of the most straightforward clustering algorithms. k centroids are first initialized randomly. Each data point is then assigned to its nearest centroid (forming an intermediate cluster). The algorithm computes the mean of the points assigned to each intermediate cluster, and repositions the centroid to this new mean. This process of assignment and centroid update is repeated until convergence (when the centroids stop moving). At this point, our final clusters have been defined (by which centroid they are nearest to)[34].

It is not uncommon for centroids to converge on less than optimal positions. We can address this by training the model multiple times, or by intelligently selecting our starting centroids. A method known as k-means++ typically performs better and converges quicker, here is how it works:

1. Select one data point at random from the dataset as the first centroid.
2. Compute the Euclidean distance from each remaining point to the nearest already-chosen centroid.

3. Select the next centroid from the remaining points with a probability proportional to the square of this distance:

$$P(x) = \frac{D(x)^2}{\sum_{x' \in X} D(x')^2}$$

where $D(x)$ is the distance from point x to the nearest chosen centroid.

4. Repeat steps 2 and 3 until k centroids have been chosen.

Step 3 still leaves some randomness in centroid initialisation, meaning we can still take multiple runs to find the most effective model.

While k-means is often a good choice, especially considering its efficiency, there are still a number of associated drawbacks.

- The clusters defined are always spherical (aside from flat hyperplanes where two clusters might intersect), meaning the model is unable to identify elongated or curved clusters.
- The model is also highly sensitive to outliers, due to the bias they can impose on the centroids, shifting them away from their optimal position, potentially causing other datapoints to be misclassified.
- K-means also requires the number of clusters k to be specified prior to training.

Gaussian Mixture Models

Gaussian Mixture Models (GMMs) take a more probabilistic approach to clustering, meaning data points have probabilities of belonging to each cluster, allowing for "soft clustering". This is done by having each cluster represented as a gaussian (normal) distribution. This allows clusters to have more nuanced shapes, as well as allowing them to overlap (with a probability given for each data point belonging to each cluster). Each cluster is defined by a mean (defining location), a covariance matrix (defining shape), and a mixing weight (defining the "importance" of each cluster).

GMMs are trained using the Expectation-Maximisation (EM) algorithm. The parameters (means, covariances, and mixing weights) are initialised randomly. Then:

- **Expectation step (E-step)**[33]: For each data point, compute the probability of it belonging to each cluster based on their current parameters.
- **Maximisation step (M-step)**[33]: Update the parameters to maximise the likelihood of the data given the probabilities calculated in the E-step.

This is repeated until the parameters converge. GMMs typically perform better than k-means, especially when working on more complex data distributions[76]. This is because their clusters can form more intricate shapes and their soft clustering techniques can give probabilities for harder to classify data point, allowing for more flexibility. They are not without drawbacks, however, aside from being much more computationally intense[76], the assumption that data has a gaussian distribution is not always the case (though they can often still model other distributions[52]). They also require the number of clusters to be specified beforehand.

DBScan

DBSCAN forms clusters based off densities in the dataset, i.e. points that are closely packed together, labelling points in harder to classify, low-density regions as outliers/noise. It also does not require the number of clusters to be specified in advance. It can also find arbitrarily shaped clusters, as it does not directly define the shape of the cluster, but merely categorises the data points. This makes it difficult to categorise new points, there are workarounds, however, as will be discussed at the end of this section.[72]

DBSCAN takes two particularly important parameters (the SKlearn library does not require them[4], but failure to define them will almost certainly leave performance on the table):

- **eps:** How close two points must be to be considered part of the same neighbourhood. Fine tuning this parameter can improve performance because if *eps* is too small, many points will be classified as noise, while if it is too large, distinct clusters are likely to merge. [72].
- **min_samples:** The number of points required within the *eps* radius to form a "dense region" or "core point" [72]. If this hyper-parameter is high, DBScan will find denser clusters, and sparser clusters if it is low.

For each data point, count how many other points are within its neighbourhood (less than *eps* distance apart). If this number is greater than "min_samples" (including itself), this can be marked as a core point, all of which now belong to the same cluster. This process continues, expanding the cluster as long as more dense regions (core points) are being found. Items more than *eps* distance away from any core point are classified as noise.[72]

Overall, DBScan is a highly effective clustering method, especially because it can form arbitrarily shaped clusters, does not require the number of clusters to be specified beforehand and can identify noise and outliers, without letting them affect the core functioning of the algorithm.

However, The two (almost) required hyperparameters *eps* and *min_samples* can make DBScan harder to use in some scenarios, sometimes requiring multiple attempts to find the best values. DBScan can also struggle in cases where the density of clusters varies (as the *eps* value is global) and for very high-dimensional data as distance metrics become less meaningful as they tend towards the same value (as points become increasingly more evenly distributed)[32][63]. Principal Component Analysis for dimensionality reduction can help with this, though it is not a complete solution[73].

HDBScan

Many of DBScan's drawbacks relating to the *eps* variable are addressed by HDBScan through the introduction of a hierarchy of density-based clusters.

This solution be thought of as searching through a range of *eps* values and picking the most robust clusters produced by them at the end. Not only does this mean *eps* does not need to be defined beforehand and fine-tuned, but it also means clusters of varying densities can be identified[7].

The algorithm picks the most robust clusters by identifying those with the most consistent membership levels[20] (conceptually for different *eps* values).

HDBScan is not without drawbacks, however. It is significantly more computationally intense, essentially running DBScan multiple times[7], as well as requiring the *min_cluster_size* hyperparameter to determine the minimum size of clusters[19].

Making Predictions with DBScan and HDBScan

Instead of defining the shape of clusters, DBScan and HDBScan merely classify the data-points they are trained on. This means they cannot natively make predictions. We can get around this, by assigning new points to the label of their nearest core point, and if there is not one, labelling the data point as noise.

The `hdbscan` library also provides an `approximate_predict()` function, though the documentation does state that it might make sense to retrain the entire model given new data points[13].

Evaluation Metrics for Clustering[71]

The robustness of clusters and the performance of clustering algorithms can be evaluated by a number of metrics.

- **Silhouette Score** quantifies how well data points have been grouped. It is calculated by the equation

$$S(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}$$

where $a(i)$ is the average distance from data point i to the other data points in the same cluster, $b(i)$ is the average distance from i to the closest data points belonging to other clusters. This equation

ranges from -1 to 1, where 1 means a point definitely belongs to its assigned cluster, 0 indicates it is on the boundary between two clusters, while -1 suggests it may be misclassified. The average Silhouette score over all data points can then be used for evaluation[71].

- **Davies-Bouldin Index**[69] quantifies both the compactness of each cluster, and how well separated they are. This is measured by this equation

$$DB = \frac{1}{k} \sum_{i=1}^k \max_{j \neq i} \left(\frac{\Delta(X_i) + \Delta(X_j)}{\delta(X_i, X_j)} \right)$$

where $\Delta(X_k)$ is the average distance between each point belonging to cluster k and the cluster's centroid, and $\delta(X_i, X_j)$ is the distance between the centroids of clusters i and j [69]. The output ranges from 0 to infinity, with higher values suggest poorer clustering[69].

2.4.6 Neural Network optimisation with Optuna

Note: The functionality of Neural Networks and Deep Learning techniques was covered extensively by Žymelka[77], so I did not include it in this research project.

To optimise neural network hyperparameter choice, I used optuna, which uses Bayesian Optimisation to explore the search space efficiently [65], avoiding a long and exhaustive grid search. By default, it uses a Tree-structured Parzen Estimator (TPE)[12] to suggest new hyperparameters from what it has learnt so far. This is done by modelling two probability distributions[11]:

- $l(x)$: good hyperparameters (those that improve performance)
- $g(x)$: bad hyperparameters

From these distributions, Optuna aims to suggest values that maximise the likelihood ratio:

$$\frac{l(x)}{g(x)}$$

Over time, $l(x)$ gets smaller as the criteria for what is considered “good” becomes stricter, allowing it to home in on optimal values.

It is crucial, however, that Optuna avoids converging its search onto “local optima” values, rather than “global optima” values. This is because the space mapping hyperparameter values onto accuracy is typically high-dimensional and non-convex[60], with numerous local “peaks”. Such local peaks can be avoided because $l(x)$ starts so large that it works similarly to random sampling. This way, as much of the hyperparameter space is search as possible before focusing on the most promising areas. It is also important to remember that $l(x)$ is a distribution, meaning it will explore hyperparameters that are similar but not identical to previous ones.

”Such local peaks can be avoided by using a sampling approach that initially broadly explores the parameter space. TPE begins with a wide acquisition function distribution ($l(x)$) that promotes exploration across the full hyperparameter space, gradually narrowing its focus to promising regions as more trials are conducted.

In addition to this, Optuna uses pruning[65] to end bad runs early if validation loss values are poor while training, so that computational resources can be focused on better hyperparameters.

Predicting the number of clusters

Predicting the number of clusters (relevant for models that require the number of clusters to be given as a hyperparameter, like k-means and GMM) in a dataset can be done by making multiple runs of these models, providing different k values each time, and then analysing different Evaluation Metrics such as inertia and Silhouette Score to see which ones form the most robust clusters.

Alternatively, there are also model-based methods:

- **X-means** for use with k-means, works out if splitting current centroids will attain a better fit by computing the Bayesian Information Criterion[40].
- **G-means** runs k-means with increasing k until it can be concluded that the data assigned to each centroid follows a gaussian distribution[39].
- **Dirichlet Process Gaussian Mixture Model** is an extension of GMM which dynamically adjusts the number of clusters based on the complexity of the data[41].

Chapter 3

Project Execution

3.0.1 Blue Pebble

I offloaded the vast majority of my computation onto Blue Pebble to save time and resources. I also used the same “short” CPU partition to maintain as much consistency as possible for time-based performance metrics. In line with best practice, I also shared a git repository between my personal computer and my Blue Pebble directory. This organised version control, and made file transfer a lot easier than using the secure copy (scp) command.

3.0.2 TensorFlow

I used TensorFlow to define my neural network architecture for its scalability, wide support, high degree of optimisation (with functions written in C++[\[50\]](#)), and ease of use. I made use of Model callbacks to automatically save models that reduced validation loss during training, so that progress would not be lost if an unexpected crash occurred. I did this in addition to saving references to which samples were used for testing so that I could guarantee that these were not used for training or validation in case I needed to test a saved model.

All neural networks were run with 1870 samples from the same participant (with 32 spectrograms for each EEG node) with 70% of the data used for training, 15% for test data, and 15% for validation data.

3.1 Methodology

3.1.1 Improving efficiency and accuracy of POS classifier

I made a number of alterations to the POS classifier produced in *Żymetka(2024)*[\[77\]](#). From these alterations, two neural networks were produced:

- One which aims to maximise efficiency.
- Another which aims to maximise accuracy.

Each test was conducted with an equal number of content and function words in the dataset and all models where time to train was measured had 90 epochs.

Of the following subsections which outline alterations, all are relevant to both, except “Conversion from image file (BMP) to list of integers (CSV)” is only relevant to the maximised efficiency model, and increasing the number of spectrogram bins, which is only relevant to the maximised accuracy model.

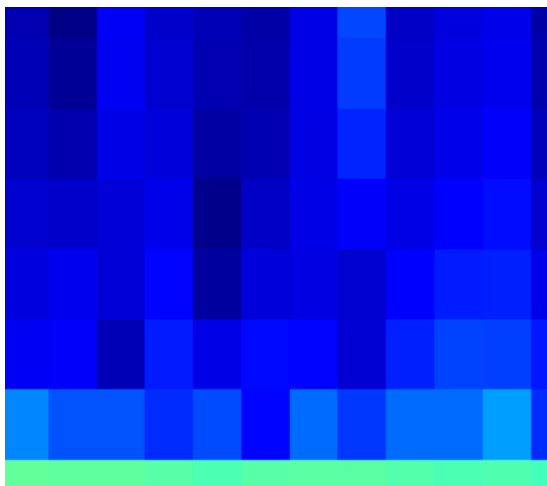
Conversion from Colour to Grayscale

The spectrogram creation MatLab code written as part of *Żymetka(2024)*[\[77\]](#) produces colour spectrograms with 96 bins (12 rows and 8 columns), and stores these as large BMP files. The colour in

spectrograms is largely for visualisation purposes, but involves storing 3 colour channels as opposed to 1. Converting this to 1 channel hugely increases efficiency as a result of:

- Lower Memory Usage, as there is a third of the data to store [62].
- Reduced computational load, as fewer computations are necessary in the convolutional layers. In CNNs, the computational load of the first convolutional layer is directly proportional to the number of input channels. [54]

It is worth noting, however, that converting from colour to greyscale is not completely lossless, as when the dimensionality of the data is reduced, fine-grained information may be lost. Typically (and in this context), each pixel in a greyscale image scales between 0 and 255, whereas in a colour image, the scale is between 0 and 255^3 (16581375). While this gives colour images a greater representational space, it is not encoding semantically meaningful information, meaning this greyscale conversion has a negligible impact on performance, with the advantage of significant computational and memory performance efficiencies. It is also worth stating that rather than converting the spectrograms from colour to greyscale, I altered the MatLab code that produces them to create them natively. 3.1 displays a before and after.



(a) Example of Original (colour) Spectrogram



(b) Example of Grayscale Spectrogram

Figure 3.1: Demonstration of Colour to Grayscale Spectrogram Conversion

Alterations to Neural Network structure

I altered the Neural Network structure to include additional dense units on each of the input branches, prior to concatenation. This allows for additional feature extraction tailored to each distinct EEG node, to learn node-specific patterns before aggregating them. This disentangles the learnt patterns, with the potential to improve accuracy. In addition to this, by reducing the dimensionality of each branch before concatenation, significantly less, but more meaningful data can be passed through to the concatenated section, as (feature extraction has already occurred). This significantly reduces memory use and compute load.

I also added a BatchNormalization layer. This losslessly transforms the data to have mean 0 and standard deviation 1 [61]. This makes the model more generalisable and typically accelerates the training process [45].

For my neural network to maximise accuracy, I also added a second convolutional layer. This improves performance by introducing Hierarchical Feature Extraction. This is where the first convolutional layer can capture lower level features, while the second builds upon this by combining these features to extract more complex patterns [21]. It also introduces Translation Invariance, where features do not have to occur in the same part of the image to be recognised[18].

Sorting of input data

The Input data I received from `Final_Code_2471654>Code>SortedData` was split into “content” and “function” folders, but conducting further analysis revealed that 35.9% words in the content folder, and 47.3% word in the function folder were misclassified. As a result, I sorted them correctly.

Optimising CNN hyperparameters

While manually testing different hyperparameters and/or neural network structures can be effective to optimise accuracy, this can be largely optimised using Python packages such as Optuna. Optuna uses Bayesian inference to efficiently search over a large hyperparameter range to find the most effective choice over this range.

Increasing the number of spectrogram bins (only relevant to accuracy model)

The spectrograms produced by “spectrogramCreation.m” are formed of 12 columns and 8 rows. To increase the granularity of the data stored in image files, I increased this to 56 columns and 107 rows.

Conversion from image file (BMP) to list of integers (CSV) (only relevant to efficiency model)

A lossless conversion from the aforementioned grayscale spectrograms (BMP files) to a compact list of 96 integers further reduced memory use drastically. As can be seen in figure 3.1, the spectrograms produced in [77] had 96 frequency bins (12*8), yet storing this as image files encodes the same bin value across hundreds of pixels, making the vast majority of the data stored redundant, so I wrote the value of each pixel into corresponding 1D CSV files, reducing the size of each file from ≈ 950 KB to just ≈ 293 bytes.

To preserve the temporal frequency structure necessary for spectrogram CNN input, I reshaped this 1D vector into a 12*8 matrix before feeding it into the model, I also reduced the kernel size to (2,2) so that it would not be overwhelmed with information covering such a large proportion of the inputs data.

3.1.2 Predicting word embeddings

The overarching structure from my content vs function classifier optimised for accuracy was a good starting point for a CNN to predict word embeddings, as I knew this methodology was more successful than others at learning complex structures and patterns from the same input data. The most obvious difference is the output, which instead of being a single sigmoid-activated node to determine class, needed to have 100 linear activation nodes to represent integers, to correspond to the vector length used by the word embedding model.

Alterations to Neural Network structure

I made the same alterations to Neural Network structure from the POS classifier to the word embedding prediction, as outlined in the section with the same title above. As well this the model underwent several rounds of tuning with Optuna.

Determining which loss function to use

An intuitive loss function for two word embedding vectors would be Mean Squared Error, which measures the average squared Euclidean distance difference between corresponding vector components. This is not appropriate, however, it assumes that magnitude carries meaningful information. In standard pretrained word embedding models such as word2vec and GloVe, magnitude is not semantically meaningful as it is positively biased by how frequently words appear in the training corpus[29].

To account for this, similarity measures must disregard magnitude; two approaches to consider are:

- Normalising the vectors before calculating Mean Squared Error: this is a valid approach in many use cases, though here it has the disadvantage of not scaling between “opposite” and “different” as clearly, as it is still inherently a measure of distance rather than direction.
- Cosine similarity: this metric directly measures the angle between two vectors (ignoring magnitude) to provide a value between -1 (completely opposite) and 1 (identical). Making this distinction between vectors that are opposite, rather than just different, facilitates more effective gradient descent by better reflecting semantic alignment in the embedding space.

As a result, I chose to use Cosine Similarity Loss, as it is most effective at capturing the semantic relationships between word embeddings.

Determining which Pre-trained Word Embedding Model to use

I chose to use the GloVe model for the following reasons:

- By using a co-occurrence matrix from the entire training corpus, GloVe encodes global relations between words, instead of just focusing on more local context as word2vec does.
- The GloVe typically, performs better on both sentiment analysis and word similarity scoring. [75]

I chose the 100-dimensional GloVe model. However, this choice introduced two notable limitations. Firstly, I chose the 100-length model to meet an adequate tradeoff between performance and computational cost, but a higher-dimensional model could capture more nuanced semantic relationships, potentially improving performance. Secondly, a static embedding model such as this one cannot account for context. For example, the word “bark” is assigned a single vector, but depending on context, it could mean a dog’s bark or tree bark. While this drawback could be addressed by using a contextual embedding model such as BERT or GPT, I was unable to do so, as despite my training data being recorded as sentences, I did not have access to it in this form. This inevitably means that some degree of accuracy is lost. Simply removing words with duplicate meanings from the dataset is also unrealistic, as 64% of words in the English language have more than one meaning depending on context [57].

Setting up Analysis

With the goal of finding out which POS tags each Part-of-Speech was most likely to be mistaken for, I formed a cosine similarity matrix between all predicted vectors and all label vectors. Using this, I hoped to find the following:

- Which words had the best predictions (where their predicted vector ranked in relation to all other labels)
- Which POSs were mostly clearly predicted from this, and/or which POS they were most likely to be confused for.

In this matrix position (i,j) holds the cosine similarity between predicted vector i and true vector j, with the corresponding words also stored to extract their POS.

3.1.3 Clustering in Word Embedding Space

I attempted a number of Clustering methods on word embedding space to see if I could uncover some underlying structure, potentially identifying new, more semantically meaningful classes, which could be used for an EEG to one-hot-encoding model. To do this, I extensively tested 4 clustering models: k-means, Gaussian-Mixture-Models, DBScan and HDBScan, the functionality of these is outlined in the Background chapter, this research informed the following methodology. In addition to the standard distance metric “euclidean”, I used cosine distance as well, equivalent to 1-cosine_similarity (reasons why this is likely a good distance metric are outlined in Section 3.1.2). I also limited the number of true word embeddings to train the model to 10,000 out of a possible 400,000. I chose this large number to limit the effect of the curse of dimensionality[63], while not performing excessive computation by using the whole pre-trained model.

K-means and Gaussian-Mixture-Models

These models require the number of clusters k to be given as an input, which was unknown, as a result, I trained the models 3 times for each k from 3 to 15, monitoring Silhouette Score and the Davies-Bouldin Index to evaluate the quality of the clusters formed. While using cosine distance would likely be more effective than euclidean distance, natively, K-means and GMM do not function well with Cosine Distance - the metric is not included in the their sklearn library implementations [6][8]. Adaptations for both do exist, such as spherical K-means clustering *K. Hornik et al. (2012)[46]* and *H. Tang et al. (2009)[42]* for GMM, but these are not as robust, for example in *H. Tang et al. (2009)[42]*, the authors state:

Although the cosine distance metric can be used in the GMM mean supervector space, it is optimal only if the data points are uniformly spread in all directions in the entire space.

As a result of these complications, I chose to instead use cosine distance with DBScan and HDBScan.

DBScan and HDBScan

These models do not require the number of clusters to be given as an input, but do require other hyperparameters to train effectively. For DBScan using the implementation provided by the sklearn library[4], these are "eps" and "min_samples", so I trained the models over [0.1, 0.2, 0.5, 0.1 2.0 3.0] and [3, 7, 15, 50, 125, 250]. For HDBScan using the hdbscan Python package[19], "min_cluster_size" is required, so I trained models over [5, 10, 25, 50, 100, 250]. These ranges were informed by the results of previous runs. Similarity to K-means, I recorded the Silhouette Score and the Davies-Bouldin Index. I also ran both models using both cosine and euclidean difference metrics.

3.1.4 Predicted Word Embeddings for Content Function Classification

An additional experiment I carried out was to determine if the predicted word embeddings produced in 3.1.2 could be used as an intermediate step, potentially benefitting from some kind of feature extraction, to content/function classification. I also kept track of a "true vector" baseline, where I tested each model on the actual vectors given my the GloVe model. This provided a baseline for how well my predicted vectors were performing.

Train/Test split For my machine learning-based methods, there were two main approaches to consider when selecting training and testing data, both offering different drawbacks and advantages:

- **Predicted embeddings for both training and testing:** This ensures that the model learns from data with the same noise and distribution as the data it will be tested on, but comes at the cost of having a limited amount of data for training.
- **True embeddings for training and predicted embeddings for testing:** While this provides as much training data as possible, this approach is not ideal as my predicted embeddings are unlikely to follow the exact the same distribution and noise patterns as the true embeddings, meaning models could be trained to predict patterns that will not actually appear when testing, leading to poor generalisation.

To balance these concerns, I opted for a compromise. For my training data I passed the original EEG training data back through the neural network, hoping to introduce the noise that can be expected from my predicted vectors as well as align the distributions more closely. This methodology also reflects the potential real deployment conditions as accurately as possible where only EEG $\rightarrow$ spectrogram $\rightarrow$ embedding is available. The following sections outline the different methodologies I employed to make content/function classifications, all GridSearches were done with 4 fold cross validation. This essentially makes use of multiple test-train splits, increasing robustness by not overfitting hyperparameter choice to a certain lucky split[23].

Cosine Similarity

The first and most simple method I employed to use word embeddings to classify content and function, was to compare each predicted vector to the average true embedding for content and function words, classifying based on which cosine similarity was greater (taking the average from my own predictions would not be a realistic use case).

Support Vector Machine (SVM)

Grid-search hyper-parameters:

Parameter	Values	Applies To
C	0.1, 1, 10	All kernels
kernel	rbf, linear, poly	N/A
degree	2, 3, 4, 5	poly
class_weight	balanced, none	All kernels

Table 3.1: Grid-search hyper-parameters for SVM

- **C:** controls the degree to which misclassified point are penalised, with a smaller value resulting in a "softer" the margin is [25].
- **class_weight**, provides the choice of whether to alter the C value for different classes (**balanced**) or not (**none**)[16].
- **kernel:** states which kernel to use:
 - **rbf** (Radial Basis Function): maps the data into an infinite-dimensional space.
Formula: $K(x, y) = e^{-\gamma \|x - y\|^2}$ [24].
 - **linear:** Used when data is linearly separable.
Formula: $K(x, y) = x \cdot y$ [24].
 - **poly:** Used to model non-linearly separable data.
Formula: $K(x, y) = (x \cdot y + c)^d$ [24]. Degree d is chosen based on the structure of the data.

Neural Network

Grid-search hyper-parameters:

Parameter	Values
hidden_layer_sizes	(64, 64), (128), (128, 64, 32), (256, 128), (512, 256, 128)
solver	adam, sgd

Table 3.2: Grid-search hyper-parameters for MLP classifier

- **hidden_layer_sizes:** I tested a number of different structures, getting results. For different numbers of layers, with different numbers of nodes.
- **solver:** The solver is the optimisation algorithm used to update weights during training[9], I chose to test:
 - **sgd:** Descends gradient by taking a sample of datapoints to make an approximation, hugely speeding up the process. Also has a fixed learning rate and is very memory efficient[22].
 - **adam:** Implements adaptive learning rate for each parameter, achieving faster convergence, and often better performance[26].

Parameter	Values
<code>max_depth</code>	None, 5, 10, 15, 20
<code>min_samples_split</code>	2, 5, 10
<code>min_samples_leaf</code>	1, 2, 4

Table 3.3: Grid-search hyper-parameters for Decision Tree classifier

Decision Tree

Grid-search hyper-parameters:

- **max_depth:** How many choices from start to leaf[1].
- **min_samples_split:** Number of samples required to cause a split[5]. This avoids over-fitting single samples that may be outliers.
- **min_samples_leaf:** Similar to the above, defines the minimum samples can be assigned to each leaf in training[5].

Random Forest

Uses the same grid-search parameters as Decision Trees, with the addition of a search over `n_estimators` (50,100,200), which is the number of Decision Trees used within the model [14].

AdaBoost

Grid-search hyper-parameters:

Parameter	Values
<code>n_estimators</code>	50, 100, 200
<code>learning_rate</code>	0.01, 0.1, 1.0, 2.0

Table 3.4: Grid search parameters for AdaBoost

- **n_estimators** The maximum number of estimators (shallow Decision Trees) to make[3].
- **learning_rate** The weight applied to each estimator with each estimation[3] this essentially scales the degree to which training data is reweighted after each round.

Voting and Stacking

Both these ensemble methods use multiple models to make a final prediction, for this reason, I chose 3 of my best models (with the best hyper-parameters found from grid-searches) from the previous sections, but not those that function too similarly (e.g. Adaboost and Random Forests), so that models can make up for each other's weaknesses. These are:

- SVM
- Neural Network
- Adaboost (performed slightly better than Decision Tree and Random Forests)

For Stacking I chose to use Logistic Regression as my meta-model because it reduces the risk of overfitting compared to more complex meta-models such as Decision Trees and Neural Networks that have the capacity to fit to noise.

Chapter 4

Results and Critical Evaluation

4.1 POS classifier

Neural network to maximise efficiency

The following results outline the efficiency improvements made by each optimisation step, measured by the time to train the model.

Model	Time to Train	Accuracy
Baseline (close to Żymelka's[77])	28 minutes 26 seconds	0.5623
Making spectrograms greyscale	18 minutes 58 seconds	0.6299
Adding dense units and batch normalization to input branches	20 minutes 7 seconds	0.5801
Conversion from BMP to CSV	1 minute 46 seconds	0.5694

Table 4.1: Models to Maximise Efficiency Training Times and Accuracy

Some of the results from 4.1 are quite unexpected. Not only did making spectrograms greyscale reduce time to train significantly, but it also had the effect of improving accuracy as well. This is likely because it reduced the model complexity as well, with fewer parameters to learn for the same amount of training data, reducing the possibility of overfitting. Furthermore, the conversion from colour to greyscale, removed colour information, that could be misleading to the model, causing it to overparameterize and learn irrelevant patterns [27]. The reduction in time to train also gives credit to this conversion.

Adding dense units and batch normalization to the input branches did not have the desired effect. The benefits I hoped to see, outlined in the methodology, were likely outweighed by the computational increase involved with performing mathematical calculations on more nodes. The change also reduced accuracy, like due to overparameterization introduced by having more nodes than necessary [27].

While making time to train almost 11* faster, conversion from BMP to CSV also reduced accuracy. Initially, I suspected this was because the model was struggling to extract features with the smaller (2,2) kernel size, so I added another convolutional layer (CONV2D) to compensate for this, but this made performance even worse. Therefore, I suspect there is something fundamentally wrong with using CSV files for the input data for TensorFlow Convolutional Neural Networks. I was unable to find any existing research into this, but I suspect it could be an issue to do with the sheer density of information, compared to an image file containing the information more sparsely as large blocks. Despite the conversion being lossless, there may also be an overparameterization risk [27], simply because there is less data to draw features from, for example, the potential to have edges and corners of each bin, may expose more information (even if irrelevant) to be extracted.

Neural network to maximise accuracy

Using spectrograms with more bins (56*107) introduced enough new data to mitigate the effects over overparameterization[27] experienced when the input data was more simple spectrograms with 12*8 bins.

Final Model	
Metric	Value
Time to train	83 minutes 41 seconds
Accuracy	0.6601

Table 4.2: Model to Maximise Accuracy Training Times and Accuracy

This meant that when I added a second convolutional layer, and more dense units to the input branch, the increased spectrogram complexity could be exploited for more accuracy, reaching 0.6601, though this came at the cost of a huge training time of 83 minutes and 42 seconds.

4.2 Predicting word embeddings

Initially, my word embedding prediction model appears to perform exceedingly well, yielding an average cosine similarity of 0.6741. Table 4.3 helps put these results into more meaningful terms.

Word 1	Word 2	Cosine Similarity
animal	pet	0.6652
food	eat	0.6302
hat	helmet	0.5720
run	walk	0.6682
in	out	0.7241
spoon	fork	0.5841
wifi	bluetooth	0.7151

Table 4.3: Cosine similarities between pairs of words

To further evaluate how accurately the predicted vectors matched the actual vectors, I used the cosine similarity matrix (the construction of which is outlined in 3.1.2) to rank the actual vectors by their cosine similarity to each predicted vector. A rank of 1 would indicate that the correct actual vector is the most similar to the predicted vector (the ideal outcome). I found the median rank between predicted and true vector to be 134 out of 281 samples, a very poor result, but still better than the completely random median ranking of 140/141, suggesting that some semantic value is still contained in these predictions. To attempt to make this conclusion, I ran the model 4 more times to get data for a one-sample t-test.

Null hypothesis H_0 : the true mean median rank is 140 (same as random prediction).

Alternative hypothesis H_1 : the true mean median rank is less than 140 (suggesting some degree of modelling).

Median ranks: 134, 131, 138, 134, 132

Sample mean = 133.8

Sample standard deviation = 2.683

Formula:

The one-sample t-test formula is [10]:

$$t = \frac{\bar{x} - \mu_0}{s/\sqrt{n}}$$

Substituting the values:

$$t = \frac{133.8 - 140}{2.683/\sqrt{5}} = \frac{-6.2}{1.200} = -5.166$$

For a significance level of 0.05 and $n - 1 = 4$ degrees of freedom, the critical t value is -2.132 .

Since -5.166 is less than -2.132 , we reject the null hypothesis.

In fact, we can reject the null hypothesis with more than 99% certainty, as the critical t value for 0.01 significance is -3.747 .

Having concluded that these results do have a lower median rank than 140, it can be deduced that the vectors do correspond with some degree of semantic meaning, as on average, they match their label more closely than a random vector. The discrepancy between a seemingly good average cosine similarity and these poor ranking results could be explained by the fact that cosine similarities can yield arbitrary and therefore meaningless results, since vectors can be scaled in certain directions arbitrarily without affecting the model's performance [43]. This explains why using cosine similarity as my evaluation metric is flawed and has yielded poor ranking results.

This poor performance is exaggerated by the overall unsuitability of using the GloVe model for my word embedding space. As it is a non-contextual model, words with multiple meanings are assigned the same vector, this disrupts the distribution of the labels of the input data, with them not corresponding to where the word, given context, should be placed in word embedding space. This issue is more pronounced than one might expect, as 64% of words in the English language have more than one meaning depending on context [57]. Resolving this with a model that accounts for context, such as BERT or GPT would definitely improve results.

4.2.1 Analysing the predictions of different Parts-of-Speech

I set out to analyse which POS tags other certain Parts-of-Speech were most likely to be mistaken for. Unfortunately, I was unable to carry out such analysis because the POS tags each POS category is mistaken for have the same distribution. The histograms below visually demonstrate this:

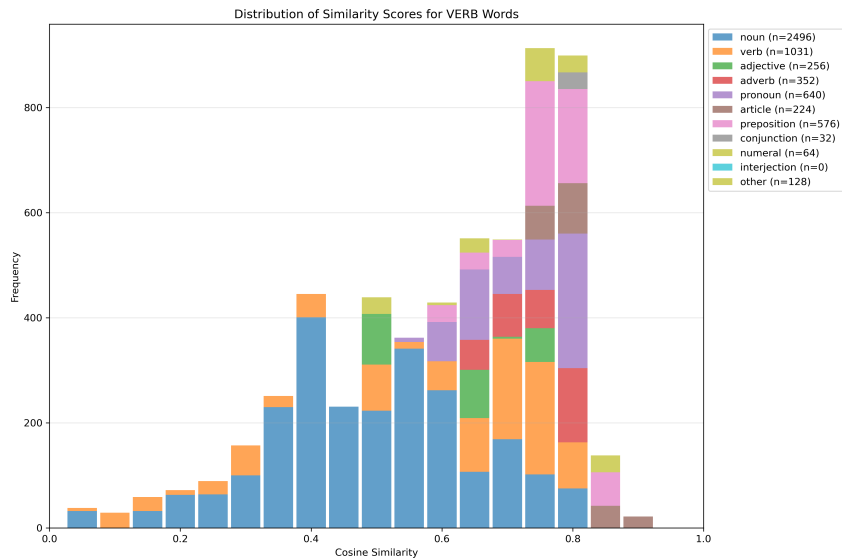


Figure 4.1: POS tags the average verb is most likely to get confused for

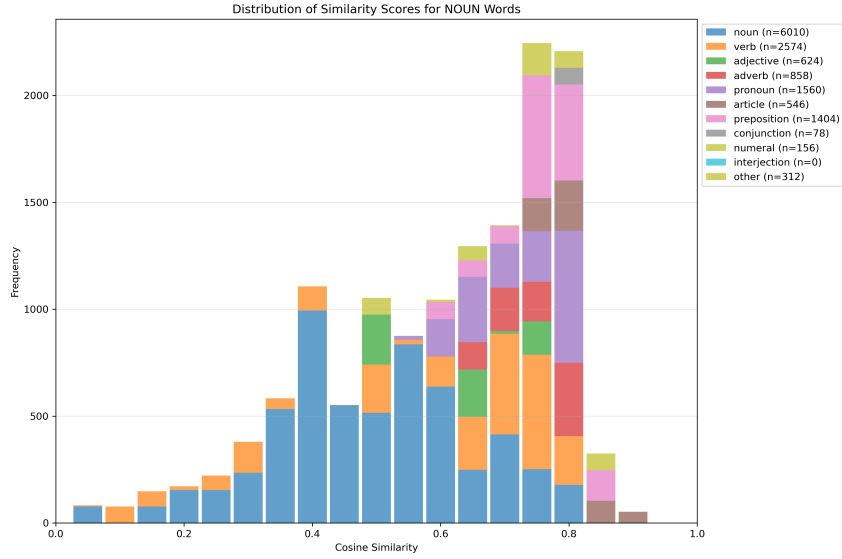


Figure 4.2: POS tags the average noun is most likely to get confused for

This averaging out of POS categories suggests very little information regarding POS is encoded in the predicted vectors, and makes the analysis I intended to carry out impossible.

4.3 Clustering in Word Embedding Space

K-means and GMM

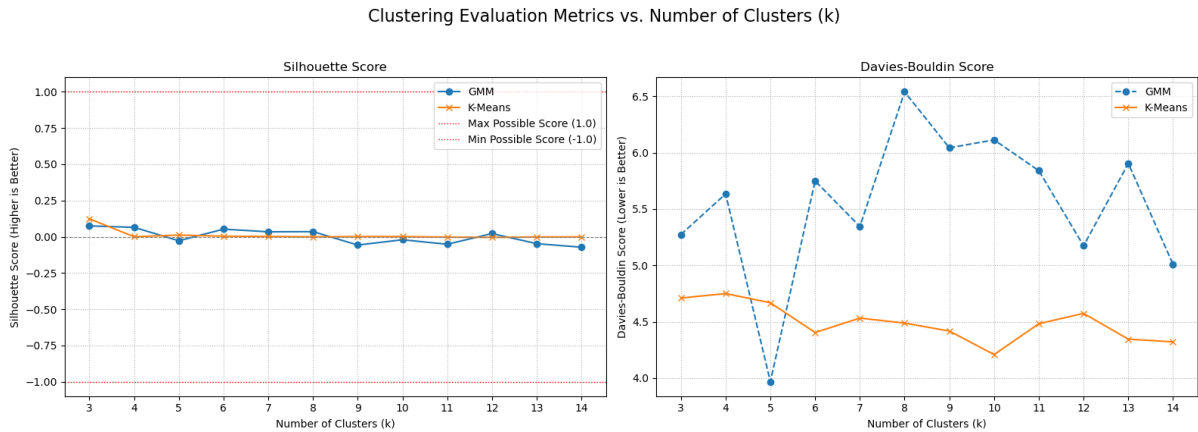


Figure 4.3: Silhouette Score (left) and Davies-Bouldin Score (right) plotted against Number of Clusters, k

Figure 4.3 displays evaluation metrics for cluster robustness, plotted against the number of clusters inputted. For both k-means and GMM, the Silhouette Score hovers very close to 0 for all numbers of clusters tested, this suggests clusters overlap heavily, with very poor structure. The Davies-Bouldin Scores found for K-means and GMM are similarly poor, an ideal score is as close to 0 as possible, but these results are significantly greater than that, with anything above 2 considered poor clustering.

Despite taking the best score produced by 3 runs, the Davies-Bouldin Scores produced by the GMM models have a lot more variance and aside from $k=5$, produced worse scores than k-means.

The poorer scores can likely be attributed GMM's sensitivity to outliers. The GMM model attempts

to classify all data points, meaning the shape of clusters may have to deviate greatly from their ideal to capture a few outlier points. While k-means is also required to classify all data-points, including outliers, clusters are only modelled by centroid position, so while outliers are likely to shift this position away from its ideal place, its entire structure does not need to be altered, as can be the case with GMM. GMM's greater variance in Davies-Bouldin Score across Number of Clusters values may be because certain numbers of clusters can deal with outliers more effectively.

This poor performance, and sensitivity to outliers was likely exaggerated by the use of Euclidean Distance, rather than Cosine Distance. The drawbacks of using Euclidean distance on this GloVe model, are outlined in "Determining which loss function to use" 3.1.2. In short, vector magnitude does not encode semantically meaningful information, distorting the word embedding space, making Cosine Distance, a far more appropriate measure. My reasons for not using cosine distance are outlined in the methodology 3.1.3. That being said, Cosine Distance can inherit the issues associated with Cosine Similarity outlined in the previous section .

DBScan

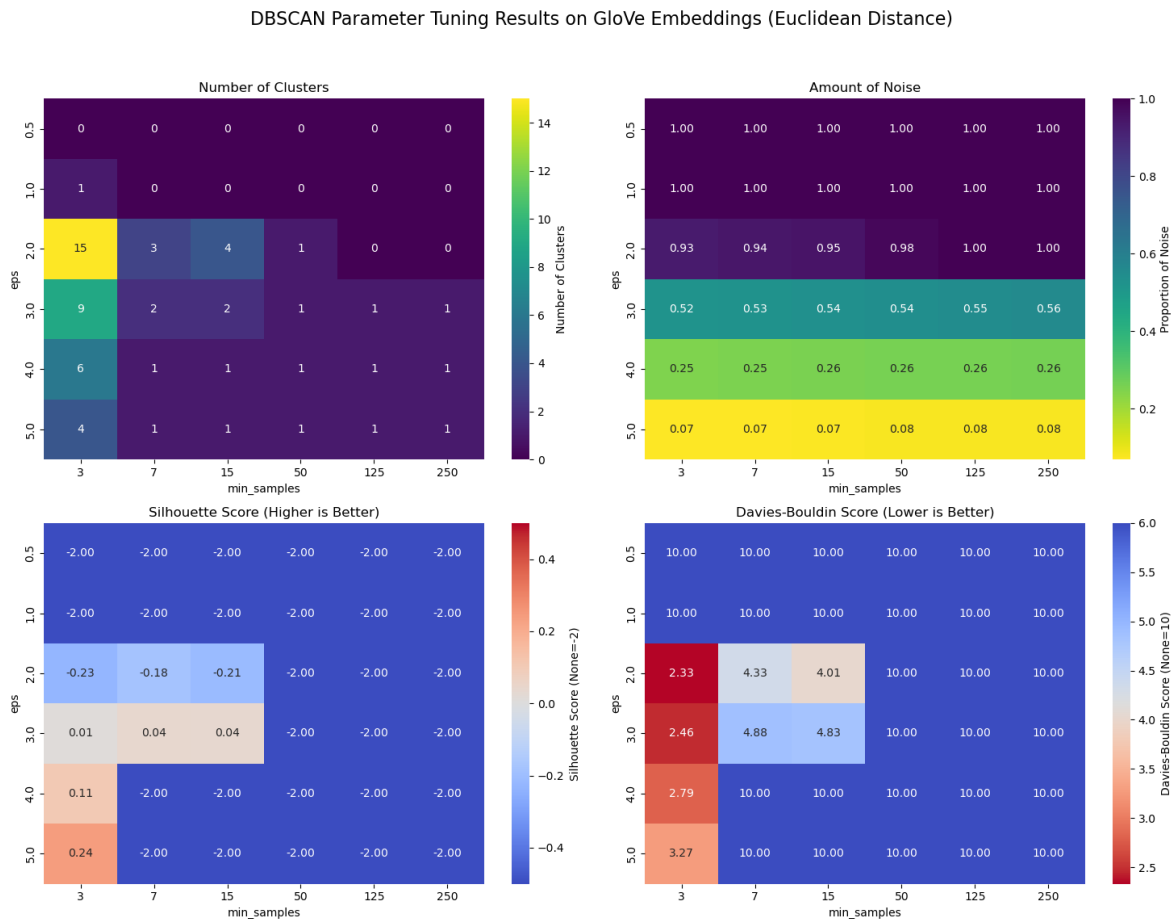


Figure 4.4: DBSCAN *min_samples* vs *eps* HeatMaps using Euclidean Distance. Top-left: Number of Clusters. Top-right: Proportion Data classified as Noise. Bottom-left: Silhouette Score, Bottom-right: Davies-Bouldin Score

I chose to use heat maps to produce 4.4 as they are effective at displaying three dimensional data. They outline the results of running DBSCAN on GloVe Embeddings using the Euclidean Distance metric, with hyperparameters plotted on the x and y axes. Outside of entries corresponding with both *min_samples* = 3 and *eps* $\geq$ 2, the model largely classifies all data points as noise, or assigns them all to one cluster. These

results demonstrate poor clustering performance, though we can make deductions about the structure of the GloVe Embedding space from them.

- **Top-Left Number of Clusters:** At small *eps* values, with *min_samples* = 3, no clusters are found, indicating this value is too strict. As *eps* increases, more clusters are found, but beyond this data points merge into a few very large clusters.
- **Top-Right Amount of Noise:** The graph shows the proportion of noise to scale from 1 (all data points classified as noise) to < 0.1 as *eps* increases and *min_samples* decreases. Cross-referencing with the Number of Clusters graph reveals that when noise classification is at a lower, more reasonable value, this is because all data points have been classified into a very small number of clusters
- **Bottom-Left and Bottom-Right** Show Silhouette Score and Davies-Bouldin score respectively. In all but 8 entries, their worst possible score is given, but some small amount of success can be seen with lower *min_samples* and *eps* values.

From these graphs, it can be deduced that for DBSCAN to produce clusters on this dataset with Euclidean Distance, **min_samples** must be low, suggesting all potential clusters are quite small. This means there is certainly some degree of separation in the structure of these data points, as it is still possible for some clusters to form, especially in the case of *eps* = 2 and *min_samples* = 3.

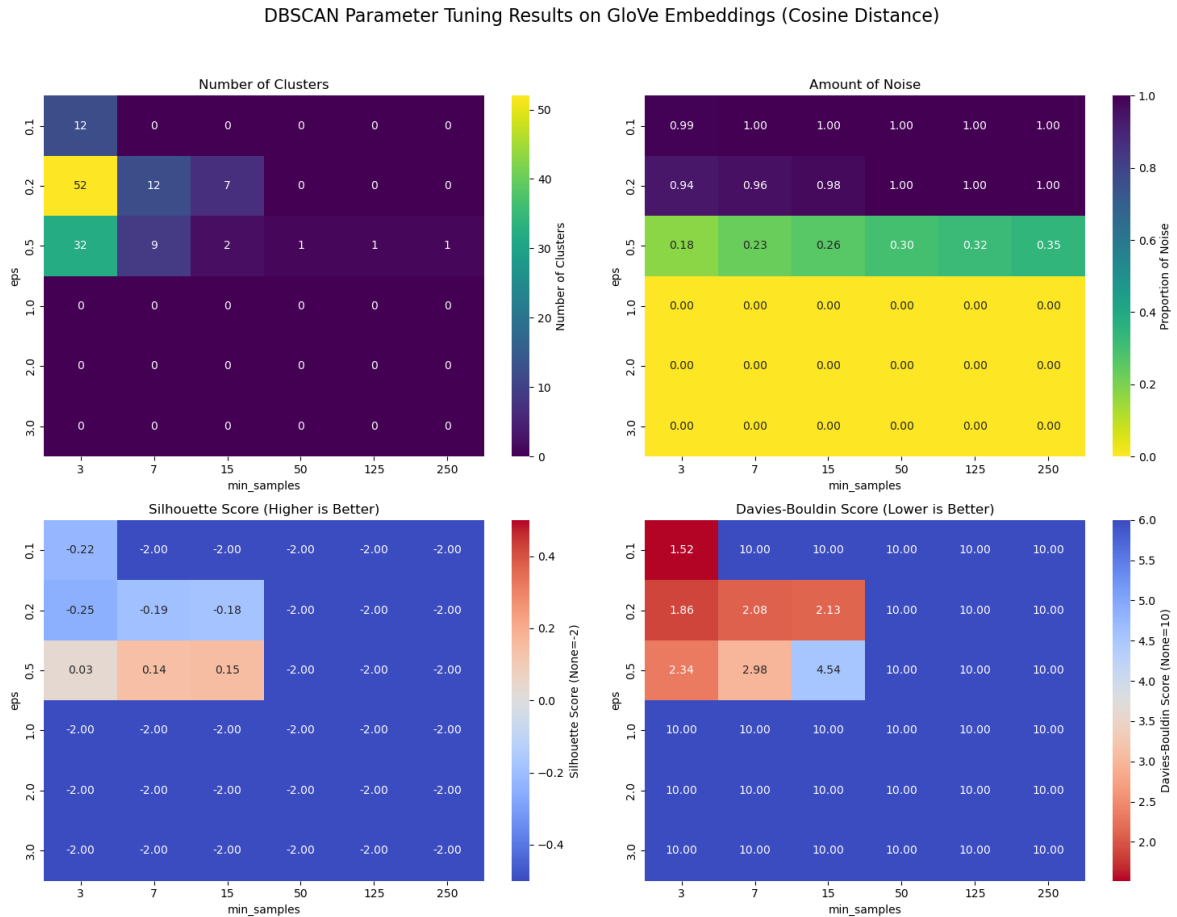


Figure 4.5: DBSCAN *min_samples* vs *eps* HeatMaps using Cosine Distance. Top-left: Number of Clusters. Top-right: Proportion Data classified as Noise. Bottom-left: Silhouette Score, Bottom-right: Davies-Bouldin Score

Using the cosine distance metric yields more robust results, as shown in 4.5. Davies-Bouldin scores

that suggest robust clusters being formed can be found where $\text{min_samples} \leq 15$ and $\text{eps} \leq 0.5$. In the case of $\text{eps} = 0.5$, these are also accompanied by a low proportion of noise classification.

Silhouette scores are less promising for these hyper parameter values (though still better than the majority of combinations). We must consider, however, that Silhouette scores are less effective at evaluating arbitrarily shaped clusters (as produced by DBScan) as it relies on the average distance from a data point to all other data points. In the case of arbitrarily shaped clusters, there are likely going to be many data points with the same classification that are a lot further than data points belonging to different clusters.

Looking at entries where evaluation metrics are best, we can see a trade off between a lot of robust clusters and a large noise classification proportion. It can certainly be said that this is the most effective clustering method so far, with $\text{min_samples} = 3$ and $\text{eps} = 0.5$ producing 32 clusters, with only 0.18 noise classification.

HDBScan

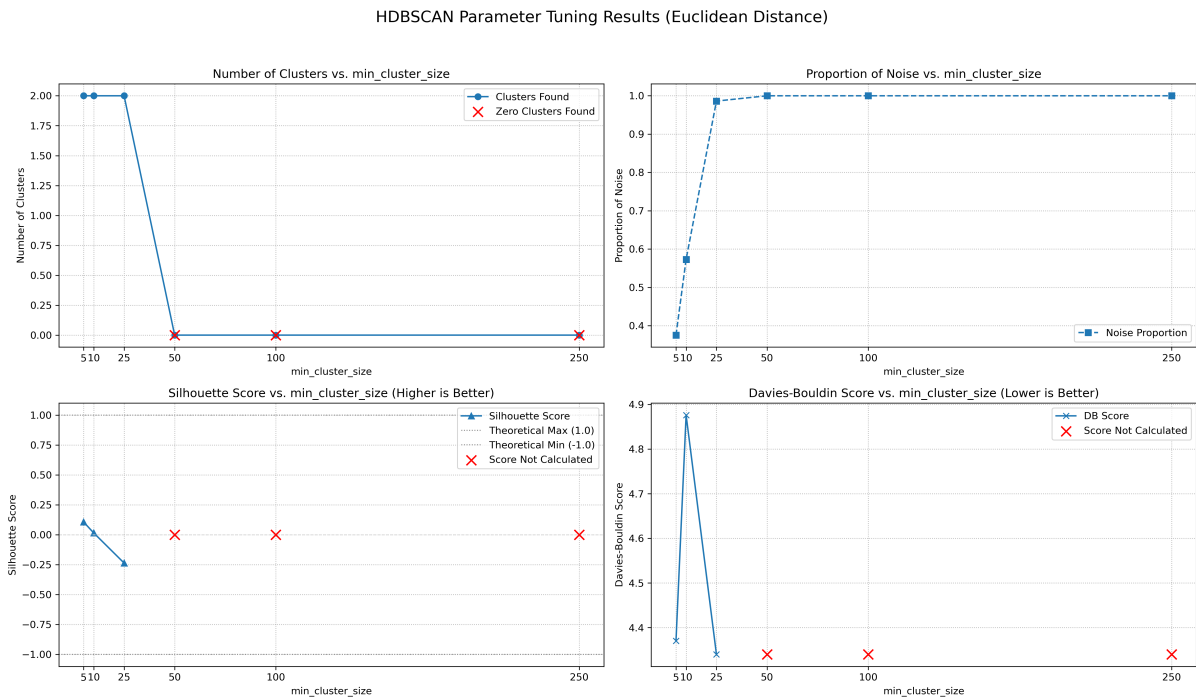


Figure 4.6: HDBScan Graphs using Euclidean Distance. Plotting `min_cluster_size` against Number of Clusters (top-left), Proportion of Noise (top-right), Silhouette Score (bottom-left), Davies Bouldin Score (bottom-right)

Figure 4.6 show overall poor results using HDBScan, both Silhouette and Davies-Bouldin scores are poor for all values, and no clusters were found above `min_cluster_size = 25`, as all data points were classified as noise. Not much can be deduced from this data, aside from the fact that HDBScan using the Euclidean Distance metric is not suitable.

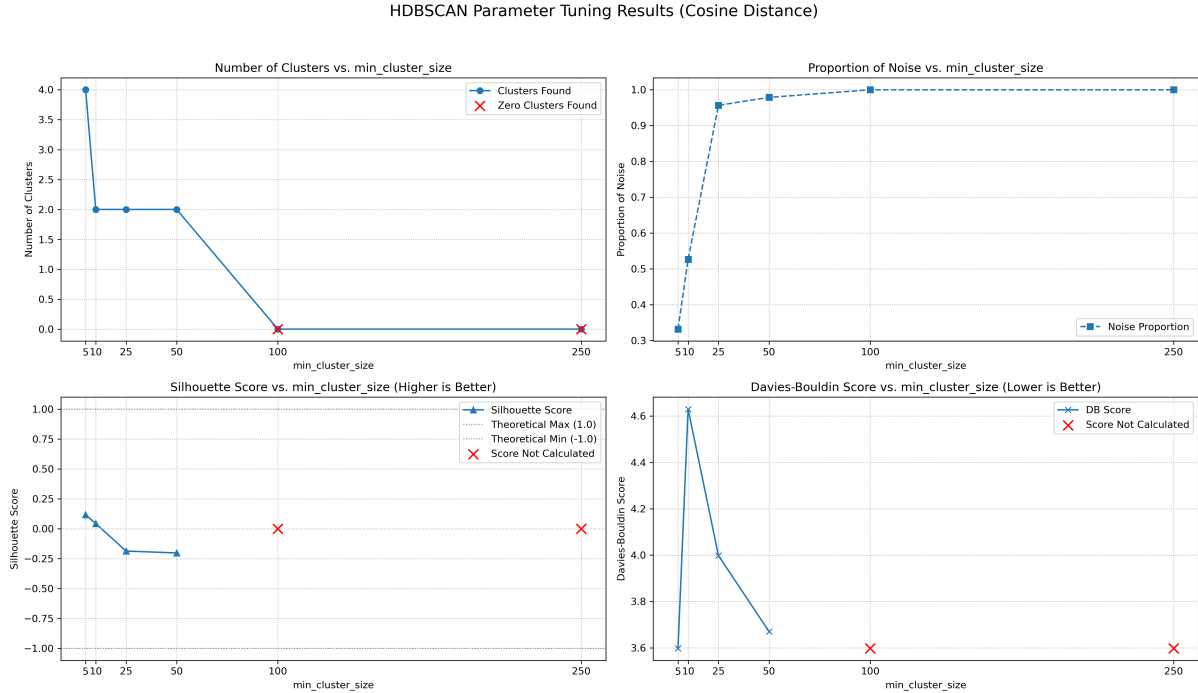


Figure 4.7: HDBScan Graphs using Cosine Distance. Plotting *min_cluster_size* against Number of Clusters (top-left), Proportion of Noise (top-right), Silhouette Score (bottom-left), Davies Bouldin Score (bottom-right)

Figure 4.7 shows a marginal performance increase from using Cosine distance over Euclidean distance for clustering, though results are still poor. More clusters were produced at higher *min_cluster_size* values compared to Euclidean distance, though their evaluation metrics do not improve.

The reduced clustering performance from HDBScan can likely be attributed to the ability to control the *eps* hyper-parameter, to favour classifying particularly close points as being in the same neighbourhood. HDBScan overrides this, aiming to find clusters based on varying densities, which can results in over-segmentation or merging of clusters that would benefit from stricter, more confined neighbourhoods.

4.3.1 Overall Analysis of Performance

None of these models performed particularly well, aside from in 4.5 with low *min_samples* and *eps* values, which show that small clusters can be found.

Clustering is clearly very challenging to carry out on this dataset, as data points are so closely positioned. Most importantly, GloVe was not trained with clustering in mind, and alternative solutions such as Latent Dirichlet Allocation (LDA)[38] and Non-Negative Matrix Factorization (NMF)[66] may be more appropriate. These are topic modelling solutions that take an entire document with the goal of deducing its topics. They have the advantage of naturally accounting for context, which GloVe does not do.

4.4 Predicted Word Embeddings for Content Function Classification

The following show results achieved on test data, alongside a baseline from true word embeddings vectors (also trained on true word embeddings vectors), as well as the best parameters from GridSearch where relevant.

Cosine Similarity

Setting	Best Parameters
Main	0.4342
Baseline	0.5231

Table 4.4: Cosine Similarity Results

This poor performance demonstrates that this is a very unsuitable method. This methodology essentially divides the word embedding space in half. The poor performance demonstrates that content and function word embeddings are spread widely across different parts of the space.

The baseline likely achieved a higher score than the main test because the average content and function vectors were taken from true word embeddings, as were the embeddings from the baseline test. The main test data was, on the other hand from a different distribution, affected by noise and transformations away from true labels, due to being predictions from a neural network.

Support Vector Machine (SVM)

Setting	Best Parameters	Accuracy
Main	{'C': 1, 'class_weight': 'balanced', 'degree': 5, 'kernel': 'poly'}	0.5587
Baseline	{'C': 10, 'class_weight': 'balanced', 'degree': 3, 'kernel': 'poly'}	0.9929

Table 4.5: SVM Grid Search Results

There were two major discrepancies between the hyperparameters found by these GridSearches, the C value determining the "softness" of the margin, and the polynomial degree the models found the most success on. Baseline found the most success with a harder margin, suggesting the boundary between content and function was more defined with true embeddings rather than predictions. This is reinforced by the difference in polynomial degrees chosen. While both models found the most success with the "poly" kernel, the baseline selected degree 3, suggesting content/function regions were more clearly defined here compared to degree 5 on the predicted vectors.

Neural Network

Setting	Best Parameters	Accuracy
Main	{'hidden_layer_sizes': 128, 'solver': 'sgd'}	0.5694
Baseline	{'hidden_layer_sizes': (256, 128), 'solver': 'adam'}	0.9929

Table 4.6: Neural Network Grid Search Results

Different hidden layer sizes suggests that too many nodes caused overfitting, especially on the predicted vector model. More were chosen for baseline because there were more valid patterns and features to learn.

Decision Tree

Setting	Best Parameters	Accuracy
Main	{'max_depth': None, 'min_samples_leaf': 1, 'min_samples_split': 2}	0.5231
Baseline	{'max_depth': 10, 'min_samples_leaf': 1, 'min_samples_split': 2}	0.9751

Table 4.7: Decision Tree Grid Search Results

Both models found the most success with *min_samples_leaf* set to 1, this is surprising as such a hyper-parameter can cause overfitting as only one sample is needed to generate a leaf node. This might explain why this produced one of the worst results for baseline, true vector performance.

Random Forest

Setting	Best Parameters	Accuracy
Main	{'max_depth': None, 'min_samples_leaf': 2, 'min_samples_split': 5, 'n_estimators': 200}	0.5267
Baseline	{'max_depth': None, 'min_samples_leaf': 1, 'min_samples_split': 10, 'n_estimators': 50}	0.9858

Table 4.8: Random Forest Grid Search Results

The parameters for Main selected by the grid search were different for the Decision Tree, selecting *min_samples_leaf* and *min_samples_split* as 2 and 5 respectively. These are considered to be better choices, as they result in a lower risk of overfitting as more than one sample is required to make a leaf or a split, stopping the model from forming its structure off single instances. These hyper-parameters may have been selected as a single Decision Tree often needs to overfit slightly to perform best, while a Random Forest aggregates multiple Decision Trees which can have numerous worse performing trees, as long as the final decision is more likely to be correct

AdaBoost

Setting	Best Parameters	Accuracy
Main	{'learning_rate': 0.1, 'n_estimators': 200}	0.5694
Baseline	{'learning_rate': 0.1, 'n_estimators': 200}	0.9893

Table 4.9: AdaBoost Grid Search Results

Both models chose the same hyperparameters. A conservative *learning_rate* of 0.1 was likely necessary to model the subtle differences between areas of content and function. Similarly, 200 estimators also reflects this subtlety.

Voting and Stacking

Best Models found from the above are given in table 4.10:

Model	Best Parameters
Neural Network	hidden_layer_sizes=128, solver='sgd'
AdaBoost	learning_rate=0.1, n_estimators=200
SVM	C=1, class_weight='balanced', degree=5, kernel='poly'

Table 4.10: Best parameters and models from previous tests

Classifier	Main Accuracy	Baseline Accuracy
Voting	0.573	0.968
Stacking	0.566	0.993

Table 4.11: Voting and Stacking Accuracy Comparison

As shown in table 4.11, the classifier that performed better was different for each data set. It is likely that stacking performed worse on Main because the Logistic Regression meta-model attempted to find patterns in errors, while in reality, there was too much noise to do so. Contrastingly, stacking performed

better for the baseline data set because it was successful in identifying the cases where different models were more likely to be correct.

The simplicity of voting likely made it more appropriate for main, because there was no ability to overfit and learn patterns that were not there. On the other hand, with baseline, it is likely that the most accurate model in each scenario was occasionally outvoted. The 3 models chosen for this test were also chosen based on performance with predicted embeddings, rather than the true embeddings with a different distribution.

4.4.1 Overall Analysis of Performance

The difference between performance on predicted (main) vs true vectors (baseline) was very stark. The very high accuracies attained on all models (besides the experimental cosine similarity method) when training and testing with true vectors suggests that content and function embeddings are indeed separated into different regions within word embedding space. This gives credit to Clustering tests 4.3, suggesting further clustering may be possible, as there are distinct patterns in the distribution of vectors to be learnt. If such patterns can be learnt to distinguish content and function, then there will certainly be more classes to learn as well.

The predicted vectors likely performed poorly because the POS prediction model produced quite poor labels. The predicted vectors were too messy to place them into clearly defined content/function regions. The fact that there were patterns to learn, however, with classification accuracy above 0.5 for all but the experimental cosine similarity test, suggests there is indeed meaningful information encoded into these predicted vectors. It must be considered though, that the Grid Searches may have, to some extent, overfit hyper parameter choice to the test data.

These results also show that that using predicted word embeddings for content/function classification was not a particularly effective method, performing worse than models that make predictions directly from EEG data 4.1. Despite this, it is possible that with a better vector prediction model this abstraction may prove effective.

Chapter 5

Conclusion

This project conducted an in depth analysis of linking EEG data to both the GloVe word embedding space and Part-of-Speech (specifically content/function classification). It largely achieved the high-level objectives set out in the introduction, improving the efficiency of the POS classifier model from *Žymelka(2024)*[77], being able to train on significantly more data with time to train less than 20 minutes [B](#). It also made accuracy far more consistent with 0.02592 standard deviation between accuracies produced in different runs?? compared to Žymelka’s 8.3028 [2.2.3](#), but ultimately never obtained the 73.33% accuracy achieved by Žymelka, falling short at 66.01%. The project also predicts word embedding vectors with an average cosine similarity of 0.6741. While this sounds extremely successful, it should also be considered that limitations in the use of the cosine similarity metric were met [\[43\]](#), with median ranking, perhaps a more realistic measure of success sitting at 134 out of 281 samples [4.3](#).

While carrying out clustering on the word embedding space ultimately did not provide useful results, I believe it is still possible given the very high accuracies achieved by the baseline tests in "Predicted Word Embeddings for Content Function Classification" [4.4](#), which suggest that content and function vectors lie in distinct areas of space. The following paragraphs will outline a number of suggestions and potential improvements that can improve and expand upon this project.

Firstly, more research and testing on image to CSV file conversion would likely yield valuable results. This project found this conversion to cause huge efficiency improvements but at the cost of consistently lower accuracy, despite the lossless conversion. I would suggest an investigation into the reasons for this, especially considering the extremely limited current research exists on this.

Secondly, attempting Principal Component Analysis (PCA) on "Predicted Word Embeddings for Content Function Classification" [4.4](#) and "Clustering in Word Embedding space" [4.3](#), may yield better results, with fewer dimensions potentially reducing complexity (though data may in turn become more noisy).

A core principle to abide by when attempting to make the fragile link between EEG data and POS/-word embeddings is to keep models simple, this project found unnecessary complexity to make models very prone to overfitting because of the overparameterization risk [\[27\]](#).

Contextual models will undoubtedly significantly improve the results of a model Predicting Word Embeddings [4.3](#) achieved when using the static GloVe model, many, if not the majority of word embeddings are to some degree inaccurate, because their semantics can differ depending on context [\[57\]](#). Also be wary of exclusively using cosine similarity for loss and evaluation metrics, *H. Steck et al. (2024)* [\[43\]](#) makes some suggestions as to how to avoid the drawbacks of cosine similarity. With an improved model, POS analysis would certainly be more insightful.

The scope of this project could be further increased by using other datasets that utilise a variety of brain recording techniques such as Magnetoencephalography (MEG) and/or Functional Magnetic Resonance Imaging (fMRI), both of which can have different Signal-to-noise ratios compared to EEG, increasing the ease of feature extraction [\[58\]](#). Additionally, monitoring intra brain communication may improve feature extraction further, a strategy of which can be found in *Mohan and Anand(2025)* [\[51\]](#).

Bibliography

- [1] 1.10. decision trees. Scikit Learn Decision Tree Documentation. URL: <https://scikit-learn.org/stable/modules/tree.html>.
- [2] 1.11. ensembles: Gradient boosting, random forests, bagging, voting, stacking. Scikit Learn Ensembles Documentation. URL: <https://scikit-learn.org/stable/modules/ensemble.html>.
- [3] Adaboostclassifier. Scikit Learn AdaBoostClassifier Documentation. URL: <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.AdaBoostClassifier.html>.
- [4] Dbscan. Scikit Learn DBScan Documentation. URL: <https://scikit-learn.org/stable/modules/generated/sklearn.cluster.DBSCAN.html>.
- [5] Decisiontreeclassifier. Scikit Learn Decision Tree Model Documentation. URL: <https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html>.
- [6] Gaussianmixture. GMM Documentation. URL: <https://scikit-learn.org/stable/modules/generated/sklearn.mixture.GaussianMixture.html>.
- [7] Hdbscan. Scikit Learn HDBSCAN Documentation. URL: <https://scikit-learn.org/stable/modules/generated/sklearn.cluster.HDBSCAN.html>.
- [8] Kmeans. Scikit Learn K-means Documentation. URL: <https://scikit-learn.org/stable/modules/generated/sklearn.cluster.KMeans.html>.
- [9] Mlpclassifier. Scikit Learn MLP Documentation. URL: https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html.
- [10] The one-sample t-test. URL: <https://www.jmp.com/en/statistics-knowledge-portal/t-test/one-sample-t-test>.
- [11] optuna.samplers.tpesampler. URL: <https://optuna.readthedocs.io/en/stable/reference/samplers/generated/optuna.samplers.TPESampler.html#optuna.samplers.TPESampler>.
- [12] optuna.study.study. URL: <https://optuna.readthedocs.io/en/stable/reference/generated/optuna.study.Study.html>.
- [13] Predicting clusters for new points. URL: https://hdbscan.readthedocs.io/en/latest/prediction_tutorial.html.
- [14] Randomforestclassifier. Scikit Learn RandomForestClassifier Documentation. URL: <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html>.
- [15] Stackingclassifier. Scikit Learn Stacking Documentation. URL: <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.StackingClassifier.html#sklearn.ensemble.StackingClassifier>.
- [16] Svc. URL: <https://scikit-learn.org/stable/modules/generated/sklearn.svm.SVC.html>.
- [17] Votingclassifier. Scikit Learn VotingClassifier Documentation. URL: <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.VotingClassifier.html>.

- [18] Translation invariance in convolutional neural networks, 2019. URL: <https://divsoni2012.medium.com/translation-invariance-in-convolutional-neural-networks-61d9b6fa03df>.
- [19] hdbscan 0.8.40, 2024. URL: <https://pypi.org/project/hdbscan/>.
- [20] Hierarchical density-based spatial clustering of applications with noise (hdbscan), 2024. URL: <https://www.geeksforgeeks.org/hdbscan/>.
- [21] Hierarchical feature learning, 2024. URL: <https://www.sapien.io/glossary/definition/hierarchical-feature-learning#:~:text=Hierarchical%20feature%20learning%20is%20a,is%20essential%20for%20accurate%20predictions>.
- [22] ML — stochastic gradient descent (sgd), 2024. URL: <https://www.geeksforgeeks.org/ml-stochastic-gradient-descent-sgd/>.
- [23] Cross validation in machine learning, 2025. URL: <https://www.geeksforgeeks.org/cross-validation-machine-learning/>.
- [24] Major kernel functions in support vector machine (svm), 2025. URL: <https://www.geeksforgeeks.org/major-kernel-functions-in-support-vector-machine-svm/>.
- [25] Support vector machine (svm) algorithm, 2025. URL: <https://www.geeksforgeeks.org/support-vector-machine-algorithm/>.
- [26] What is adam optimizer?, 2025. URL: <https://www.geeksforgeeks.org/adam-optimizer/>.
- [27] Yuanzhi Li Ahmet Cagri Duzgun, Samy Jelassi. How does overparameterization affect features? *arXiv:2407.00968v1 [cs.LG]*, 2024. URL: <https://arxiv.org/html/2407.00968v1>.
- [28] Heike Behrens. Constructivist approaches to first language acquisition. *Journal of Child Language*, 2021. URL: <https://www.cambridge.org/core/journals/journal-of-child-language/article/constructivist-approaches-to-first-language-acquisition/9859E93FA9AA8D32F269DE38E721EEA6>.
- [29] Adriaan M. J. Schakel Benjamin Wilson. Controlled experiments for word embeddings. 2015.
- [30] R. Berwick. An idiot’s guide to support vector machines (svms), unknown.
- [31] Fernando Cuetos Carolina Yudes, Alberto Domínguez and Manuel de Vega. The time-course of processing of grammatical class and semantic attributes of words: Dissociation by means of erp. *Psicológica*, 2016. URL: <https://www.redalyc.org/articulo.oa?id=16946248001>.
- [32] Avi Chawla. The limitations of dbscan clustering which many often overlook, 2023. URL: <https://blog.dailydoseofds.com/p/the-limitations-of-dbscan-clustering>.
- [33] James Cussens. The em algorithm, 2024. University of Bristol - Machine Learning Lecture notes.
- [34] James Cussens. k-means and mixtures of gaussians, 2024. University of Bristol - Machine Learning Lecture notes. URL: <https://uob-coms30035.github.io/lectures/lect11/lect11.pdf>.
- [35] James Cussens. Kernels and support vector machines, 2024. University of Bristol - Machine Learning Lecture notes. URL: <https://uob-coms30035.github.io/lectures/lect08/lect08.pdf>.
- [36] James Cussens. Regression and classification trees, 2024. University of Bristol - Machine Learning Lecture notes. URL: <https://uob-coms30035.github.io/lectures/lect07/lect07.pdf>.
- [37] Sahil Datta and Nikolaos V. Boulgouris. Recognition of grammatical class of imagined words from eeg signals using convolutional neural network. *Neurocomputing*, 2021. URL: <https://www.sciencedirect.com/science/article/pii/S09255231221012194#:~:text=In%20order%20to%20recognise%20imagined,class%20classification%20phonemes%20and%20words>.
- [38] Andrew Y. Ng David M. Blei and Michael I. Jordan. Latent dirichlet allocation. *Journal of Machine Learning Research* 3, 2003. URL: <https://www.jmlr.org/papers/volume3/blei03a/blei03a.pdf>.
- [39] Charles Elkan Greg Hamerly. Learning the k in k-means. 2003. URL: https://proceedings.neurips.cc/paper_files/paper/2003/file/234833147b97bb6aed53a8f4f1c7a7d8-Paper.pdf.

- [40] Aman Gupta. X-means — a complement to the k-means clustering algorithm, 2021. URL: <https://medium.com/geekculture/x-means-algorithm-a-complement-to-the-k-means-algorithm-b087ae88cf88>.
- [41] Dilan Görür¹ and Carl Edward Rasmussen. Dirichlet process gaussian mixture models: Choice of the base distribution. *JOURNAL OF COMPUTER SCIENCE AND TECHNOLOGY*, 2010. URL: <https://mlg.eng.cam.ac.uk/pub/pdf/GoeRas10.pdf>.
- [42] Thomas S. Huang Hao Tang, Stephen M. Chu. Spherical discriminant analysis in semi-supervised speaker clustering. *Proceedings of NAACL HLT*, 2009.
- [43] Chaitanya Ekanadham Harald Steck and Nathan Kallus. Is cosine-similarity of embeddings really about similarity? 2024. URL: <https://arxiv.org/pdf/2403.05440v1>.
- [44] Richard Socher Jeffrey Pennington and Christopher D. Manning. Glove: Global vectors for word representation. pages 1532–1543, 2014. URL: <https://aclanthology.org/D14-1162/>.
- [45] Nico Klingler. The role of batch normalization in cnns. URL: <https://viso.ai/deep-learning/batch-normalization/#::~text=Batch%20normalization%20aims%20to%20improve,will%20accelerate%20the%20training%20process>.
- [46] Martin Kober Kurt Hornik, Ingo Feinerer and Christian Buchta. Spherical k-means clustering. *Journal of Statistical Software*, 2012.
- [47] Stephen Laurence and Eric Margolis. The poverty of the stimulus argument. *The British Journal for the Philosophy of Science*, 2001. URL: <https://philarchive.org/archive/LAUTPO>.
- [48] Paul Main. Chomsky’s theory, 2023. URL: <https://www.structural-learning.com/post/chomskys-theory>.
- [49] M Brandon Westover Marcus C Ng, Jin Jing. A primer on eeg spectrograms. *J Clin Neurophysiol*, 2022. URL: <https://pubmed.ncbi.nlm.nih.gov/34510095/>.
- [50] Jianmin Chen Zhifeng Chen Andy Davis Jeffrey Dean Matthieu Devin Sanjay Ghemawat Geoffrey Irving Michael Isard Manjunath Kudlur Josh Levenberg Rajat Monga Sherry Moore Derek G. Murray Benoit Steiner Paul Tucker Vijay Vasudevan Pete Warden Martin Wicke Yuan Yu Xiaoqiang Zheng Martín Abadi, Paul Barham. Tensorflow: A system for large-scale machine learning. 2016. URL: <https://arxiv.org/abs/1605.08695>.
- [51] Anand Mohan and R. S. Anand. Classification of imagined speech signals using functional connectivity graphs and machine learning models. *Brain Topography*, 2025. URL: <https://link.springer.com/content/pdf/10.1007/s10548-025-01100-7.pdf>.
- [52] Tilak Mudgal. Gaussian mixture modeling (gmm), 2024. URL: <https://medium.com/@tilak559/gaussian-mixture-modeling-gmm-1cb5f00a97d9#:::text=Imagine%20you%20have%20a%20dataset,Clustering%2C%20and%20Kernel%20Density%20Estimation>.
- [53] Chetan S. Nayak and Arayamparambil C. Anilkumar. Eeg normal waveforms. 2023. URL: <https://www.ncbi.nlm.nih.gov/books/NBK539805/>.
- [54] Joseph Nelson, 2020. URL: <https://blog.roboflow.com/when-to-use-grayscale-as-a-preprocessing-step/>.
- [55] Normalized Nerd. Introduction to nlp — word embeddings word2vec model, 2020. Accessed: 2025-04-18. URL: https://www.youtube.com/watch?v=_Rt4Ljas034&ab_channel=NormalizedNerd.
- [56] Benjamin Glaus Maria Barrett Marius Troendle Nicolas Langer Nora Hollenstein, Cedric Renggli and Ce Zhang. Decoding eeg brain activity for multi-modal natural language processing. *Front. Hum. Neurosci*, 2021. URL: <https://www.frontiersin.org/journals/human-neuroscience/articles/10.3389/fnhum.2021.659410/full>.
- [57] Meral Ozturk. *International Journal of Curriculum and Instruction*, 2017. URL: <https://eric.ed.gov/?id=EJ1207245>.

- [58] Vorwerk J Clerc M Engwer C Piastra MC, Nüßing A and Wolters CH. A comprehensive study on electroencephalography and magnetoencephalography sensitivity to cortical and subcortical sources. *Hum Brain Mapp*, 2020. URL: <https://pmc.ncbi.nlm.nih.gov/articles/PMC7856654/#:~:text=From%20Figure%20a%2C%20we%20can,or%20less%20similar%20SNR%20values>.
- [59] F. Mormann C. Rieke P. David R. G. Andrzejak, K. Lehnertz and C. E. Elger. The erp response to the amount of information conveyed by words in sentences. *Brain and Language*, 140:1–11, 2015. URL: <https://www.sciencedirect.com/science/article/pii/S0093934X14001515>.
- [60] Karthik Rangasai. How do you pick the right set of hyperparameters for a machine learning project? URL: <https://devblog.pytorchlightning.ai/how-do-you-pick-the-right-set-of-hyperparameters-for-a-machine-learning-project-975951644152>.
- [61] Shipra Saxena. Introduction to batch normalization. URL: <https://www.analyticsvidhya.com/blog/2021/03/introduction-to-batch-normalization/>.
- [62] J.O. Schnepapat. URL: <https://schnepapat.com/grayscale.html>.
- [63] Badreesh Shetty. What is the curse of dimensionality?, 2022. URL: <https://builtin.com/data-science/curse-dimensionality#:~:text=Curse%20of%20Dimensionality%20in%20Distance%20Function&text=In%20other%20words%2C%20as%20the,between%20data%20points%20the%20same.&text=From%20the%20above%20figure%2C%20it,while%20also%20economizing%20on%20memory>.
- [64] Manan Suri. A dummy’s guide to word2vec. 2022. URL: <https://medium.com/@manansuri/a-dummys-guide-to-word2vec-456444f3c673>.
- [65] ODSC Team. Optuna: An automatic hyperparameter optimization framework. URL: <https://odsc.com/blog/optuna-an-automatic-hyperparameter-optimization-framework/>.
- [66] Jaegul Choo Tian Shi, Kyeongpil Kang and Chandan K. Reddy. Short-text topic modeling via non-negative matrix factorization enriched with local word-context correlations. *International World Wide Web Conferences Steering Committee*, 2018. URL: <https://dl.acm.org/doi/abs/10.1145/3178876.3186009>.
- [67] Ngoc Quach Tianhao Wu, Yu Wang. Advancements in natural language processing: Exploring transformer-based architectures for text understanding. 2025. URL: <https://arxiv.org/abs/2503.20227v1>.
- [68] Greg Corrado Jeffrey Dean Tomas Mikolov, Kai Chen. Efficient estimation of word representations in vector space. *arXiv:1301.3781*, 2013. URL: <https://arxiv.org/abs/1301.3781>.
- [69] unknown. Davies-bouldin index, 2023. URL: <https://www.geeksforgeeks.org/davies-bouldin-index/>.
- [70] unknown. Bagging vs boosting in machine learning, 2025. URL: <https://www.geeksforgeeks.org/bagging-vs-boosting-in-machine-learning/>.
- [71] unknown. Clustering metrics in machine learning, 2025. URL: <https://www.geeksforgeeks.org/clustering-metrics/>.
- [72] unknown. Dbscan clustering in ml — density based clustering, 2025. URL: <https://www.geeksforgeeks.org/dbscan-clustering-in-ml-density-based-clustering/>.
- [73] unknown. Principal component analysis(pca), 2025. URL: <https://www.geeksforgeeks.org/principal-component-analysis-pca/>.
- [74] Valerie Davis R.EEG.T CLTM William O Tatum IV DO FACNS, Anteneh M Feyissa MD MSc. Technical aspects of eeg. 2023. URL: <https://www.medlink.com/articles/technical-aspects-of-eeg>.
- [75] Amit Yadav, 2024. URL: <https://medium.com/biased-algorithms/word2vec-vs-glove-which-word-embedding-model-is-right-for-you-4dfc161c3f0c>.

- [76] Amit Yadav. K means clustering vs gaussian mixture, 2024. URL: <https://medium.com/@amit25173/k-means-clustering-vs-gaussian-mixture-bec129fbe844>.
- [77] Klaudia Żymelka. Classifying neurolinguistics using deep learning architecture on eeg spectrograms. School of Computer Science IRP, University of Bristol, 2024.

Appendix A

Appendix A: AI Prompts/Tools

The following figures were produced with the assistance of ChatGPT for producing Python Code: [4.1](#), [4.2](#), [4.3](#), [4.4](#), [4.5](#), [4.6](#), [4.7](#).

Appendix B

Raw Test Results

Try	Accuracy	Time (HH:MM:SS)
1	0.5480	00:17:56
2	0.5801	00:19:20
3	0.6299	00:18:58
4	0.6085	00:17:44
5	0.5623	00:14:14
6	0.5658	00:16:14
7	0.5730	00:17:34
8	0.5623	00:17:38
9	0.5445	00:15:09
9	0.5516	00:15:03

Table B.1: Accuracy and Duration for 10 runs of Second Model from [4.1](#)